

Курсовая работа защищена с оценкой _____

Учёный секретарь кафедры _____

Московский государственный университет имени М. В. Ломоносова
Механико-математический факультет
Кафедра вычислительной математики

КУРСОВАЯ РАБОТА

Прогнозирование временных рядов: исследование символьного и численного представления тиков

Выполнена студентом 341 группы
Журавлёвым Георгием Валерьевичем

Научный руководитель:

д.ф.-м.н. М. И. Кумсков

Москва, 2026

Аннотация

Работа содержит 52 страницы, 19 таблиц, 5 рисунков, 48 источников использованной литературы.

Объект исследования — задача прогнозирования и классификации временных рядов на двух качественно различных доменах (финансовый ряд BTC/USDT и архив UCR Time Series Classification).

Цель — сравнить численное и символьное представления временного ряда и установить, в каких ситуациях каждое из них предпочтительнее.

Методы — классический Transformer-encoder и PatchTST на относительных свечных признаках; SAX, SAX Bag-of-Words, BOSS Ensemble (словарный символьный метод на основе символьного Фурье-приближения SFA); специализированные модели прогнозирования NeuralForecast (DLinear, NLinear, NHITS, NBEATSx, TimesNet, TFT, TiDE, iTransformer); foundation-модель Chronos Bolt Small в режиме zero-shot; сравнение с тривиальными baseline (Naive, zero_return, majority_sign).

Данные — BTC/USDT на трёх частотах (1m, 1h, 1d) с горизонтами 1–24; 8 уникальных датасетов архива UCR (Coffee, GunPoint, Trace, TwoLeadECG, ECG5000, FordA, Wafer, ElectricDevices) в 9 экспериментальных постановках.

Главный результат — на выбранных UCR-датасетах три метода (random-convolutional ROCKET и MiniRocket, символьный BOSS) образуют верхнюю группу со статистически неотличимым качеством (Friedman $p = 1,8 \cdot 10^{-7}$), значимо опережая обучаемый Transformer на сыром численном сигнале и наивные baseline. BOSS — лучший среди именно символьных методов, но в среднем уступает random-convolutional MiniRocket, который при этом на 2–3 порядка быстрее. На BTC в проведённых экспериментах не обнаружено статистически подтверждённого устойчивого преимущества обучаемых моделей над тривиальным Naive-прогнозом ни на одной из проверенных конфигураций (Wilcoxon $p = 0,008$, win/loss 0/8 для каждой из пяти современных архитектур).

Практическая значимость — получен количественный критерий применимости символьного представления: оно полезно при наличии воспроизводимой локальной морфологии и/или малой обучающей выборки; на финансовых рядах с высокой долей случайности оно не даёт преимущества и должно сопоставляться с тривиальными baseline.

Ключевые слова: временные ряды, прогнозирование, символьное представление, BOSS, SAX, японские свечи, Transformer, PatchTST, foundation-модели.

Содержание

Введение	5
1 Теоретические основы	8
1.1 Временные ряды и постановка прогнозирования	8
1.2 Тривиальные baseline	8
1.3 Архитектура Transformer	9
1.4 Специализированные Transformer-модели для прогнозирования	9
1.5 Японские свечи	10
1.6 Символьное представление временных рядов	10
1.7 Вычислительные ограничения и переобучение	11
1.8 Финансовые ряды, случайное блуждание и эффективный рынок	11
2 Постановка задачи и используемые данные	13
2.1 Формальная постановка	13
2.2 Используемые наборы данных	14
2.3 Протокол обучения и валидации	16
2.4 Используемые модели и инфраструктура	17
3 Численное представление и Transformer на относительных свечах	17
3.1 Признаковое описание одной свечи	17
3.2 Восстановление цены из прогноза доходности	18
3.3 Архитектура численного Transformer	18
3.4 Результаты на BTC: регрессия цены	19
3.5 Результаты на BTC: классификация направления	20
3.6 Эксплоративные эксперименты	20
3.7 Промежуточный вывод	21
4 Символьное представление: SAX, BOSS и токены свечей	21
4.1 Метод SAX и SAX+Embedding+Transformer	21
4.2 SAX Bag-of-Words и логистическая регрессия	22
4.3 Метод BOSS	22
4.4 Ручная символьная кодировка свечей на BTC	23
4.5 Символьный Transformer на BTC	23
5 Доменная зависимость полезности символьного представления	23
5.1 UCR, малые датасеты	24
5.2 Попытка выровнять PatchTST: аугментация x5	26
5.3 UCR, крупные датасеты	27
5.4 Устойчивость к шуму: ECG5000 + Gaussian noise	28
5.5 Сравнение BOSS с InceptionTime	29
5.6 BTC: расширенное сравнение современных моделей прогнозирования	29

5.7	ВТС: классификация направления	32
5.8	Когда символьное представление работает	34
6	Дополнительные исследования	35
6.1	Статистическая проверка различий	35
6.2	Вычислительная сложность	37
6.3	Краткий обзор WEASEL и HIVE-COTE	39
6.4	Foundation-модели для временных рядов как направление развития . . .	40
7	Ограничения работы	41
	Заключение	42
A	Репликационный пакет	49
B	Используемые версии библиотек	51
C	Список сокращений	51

Введение

Прогнозирование и классификация временных рядов — одна из классических задач прикладной математики и машинного обучения. Под временным рядом понимается упорядоченная по времени последовательность наблюдений $\{x_t\}_{t=1}^T$, где значение x_t может быть скалярным или векторным. Задачи могут ставиться как восстановление будущих значений ряда (прогноз), как предсказание знака приращения на горизонте h , так и как присвоение метки классу всего ряда длины L (классификация ряда целиком).

Постановка задачи внешне универсальна, однако её содержательная сложность существенно зависит от конкретного домена. В настоящей работе рассматриваются два качественно различных домена.

Финансовые ряды. Цены биржевых активов на коротких горизонтах близки к процессу с высокой долей случайности. Это эмпирически проявляется в том, что простейшие тривиальные прогнозы (например, `Naive`, `zero_return`, `majority_sign`) оказываются очень сильными baseline-моделями. Любое улучшение прогноза, измеряемое по средней абсолютной ошибке регрессии, имеет порядок единиц процентов и должно сопоставляться именно с такими baseline-моделями.

Ряды с воспроизводимой морфологией. Сюда относятся биомедицинские сигналы (ЭКГ, ЭЭГ), показания датчиков машин и электроприборов, спектроскопические данные. Такие ряды содержат локально воспроизводимую форму, обусловленную физическим механизмом порождения сигнала. Стандартным открытым тестовым полигоном для них служит архив UCR Time Series Archive [45].

Это различие напрямую влияет на то, какое представление входных данных оказывается полезным.

Численное представление — последовательности относительных численных признаков (нормированные приращения цены, инженерная геометрия свечи: тело, тени, диапазон, расширенный стек `full_stack` с объёмом, спредом и волатильностью). Численное представление сохраняет полную информацию о состоянии ряда, но передаёт её модели в виде непрерывных значений и потому требует достаточного количества обучающих данных и аккуратной нормировки.

Символьное представление — последовательности дискретных токенов, получаемых из ряда либо путём ручной символьной кодировки (на финансовых данных — кодировка японских свечей через направление, размер тела, длины теней и режим объёма), либо алгоритмически. Алгоритмическим эталоном для классификации временных рядов является метод BOSS [31]: ряд символизируется через символьное Фурье-приближение SFA, после чего классификатор работает с гистограммами символьных слов. BOSS известен тем, что без какого-либо обучения нейронной сети и на CPU достигает качества, конкурентного современным глубоким ансамблям [34].

Исследовательские гипотезы.

- **H1.** На временных рядах с воспроизводимой локальной морфологией символьные

словарные методы оказываются эффективнее малых нейросетевых моделей при малом числе обучающих примеров.

- **Н2.** На краткосрочных финансовых рядах BTC символьная свечная кодировка не даёт устойчивого преимущества над численным представлением и тривиальными baseline.
- **Н3.** Преимущество символизации зависит не от дискретности как таковой, а от совпадения дискретных «слов» с повторяющейся локальной структурой сигнала; при её отсутствии словарное представление теряет информативность.

В работе различаются три постановки задачи: *forecasting* (прогноз будущих значений ряда; применяется к BTC), *direction classification* (предсказание знака приращения; применяется к BTC) и *whole-series classification* (присвоение метки классу всего ряда длины L ; применяется к UCR). Выводы о роли представления формулируются отдельно для каждой постановки и не переносятся между ними автоматически.

Цель работы. Сравнить численное и символьное представления временных рядов в указанных трёх постановках и установить, в каких ситуациях каждое из них предпочтительнее.

Научная новизна. Работа не предлагает новой нейросетевой архитектуры. Её содержательный вклад состоит в трёх элементах:

1. *Систематическое сопоставление* численного и символьного представлений на двух доменах с принципиально различной природой сигнала (финансовый ряд BTC/USDT и архив UCR), при едином протоколе сравнения и единых тривиальных baseline.
2. *Экспериментальная формулировка критерия применимости символизации*: символьное представление полезно при наличии воспроизводимой локальной морфологии сигнала и/или малой обучающей выборки; на близких к случайному блужданию рядах (краткосрочный BTC) оно не даёт устойчивого преимущества и должно сопоставляться с тривиальным Naive-прогнозом.
3. *Статистически подтверждённый отрицательный результат на BTC*: ни одна из проверенных современных архитектур прогнозирования временных рядов (DLinear, NLinear, NHITS, NBEATSx, PatchTST, TimesNet, TFT, TiDE, iTransformer, FuzzyLSTM, Chronos Bolt Small, символьный Transformer) не превосходит Naive-прогноз ни на одной из проверенных конфигураций (Wilcoxon $p = 0,008$ против Naive). Контрольный backtest показывает, что слабая статистическая значимость направленного прогноза (hit rate 0,53–0,54) не покрывает транзакционных издержек.

Задачи работы:

1. Описать математические основы рассматриваемых представлений (символьная агрегация SAX, символьное Фурье-приближение SFA и метод BOSS, токенизация

японских свечей) и моделей, применяемых поверх них (классический Transformer-encoder, PatchTST, специализированные архитектуры из NeuralForecast, foundation-модели семейства Chronos).

2. Реализовать прогнозную систему для финансового ряда BTC/USDT с двумя представлениями и сравнить её результаты с расширенным набором baseline и современных моделей прогнозирования на трёх частотах данных (минутные, часовые, дневные).
3. Реализовать символьную классификацию на 8 уникальных датасетах архива UCR (Coffee, GunPoint, Trace, TwoLeadECG, ECG5000, FordA, Wafer, ElectricDevices; в сумме 9 экспериментальных постановок — TwoLeadECG в двух режимах) и сравнить четыре подхода: численный PatchTST, символьный SAX+Embedding+Transformer, словарный символьный SAX-BoW+LogReg и BOSS Ensemble.
4. Проверить, насколько преимущество символьных методов на малой выборке закрывается стандартной аугментацией для временных рядов (jittering, scaling, time warping) с пятикратным увеличением обучающего набора.
5. Сопоставить поведение символьных и численных моделей на финансовом домене и на доменах с воспроизводимой локальной морфологией; сформулировать выводы о доменной зависимости полезности символьного представления.

Методы. Используются: классический Transformer-encoder и PatchTST на PyTorch [2]; метод SAX через библиотеку `tslearn` [43] и SAX-BoW через `pyts` [44]; реализация BOSS Ensemble из библиотеки `aeon` [42]; специализированные модели прогнозирования через NeuralForecast [22]; foundation-модель ChronosBoltSmall [24] в режиме zero-shot; FuzzyLSTM [23]; протокол walk-forward валидации для финансовых рядов и стандартные предопределённые UCR-разбиения для классификации; стандартные метрики MAE, RMSE, sMAPE, accuracy, balanced accuracy, F_1 . Все эксперименты воспроизводимы по приложенным ноутбукам и сохранённым CSV/JSON-файлам с результатами; репликационный пакет опубликован: <https://github.com/qqsta37/symbolic-vs-numeric-timeseries>.

Структура работы. В разделе 1 приведены теоретические основы. В разделе 2 формулируются задачи, описываются используемые наборы данных и модели. В разделе 3 разбирается численное представление и Transformer на относительных свечных признаках. В разделе 4 описаны символьные представления (ручная кодировка свечей, SAX, SFA и BOSS) и соответствующие модели. В разделе 5 проводится систематическое сравнение по доменам и обсуждается основной результат работы. В заключении сформулирован итоговый вывод.

1 Теоретические основы

1.1 Временные ряды и постановка прогнозирования

Под временным рядом понимается последовательность $X = (x_1, \dots, x_T)$, $x_t \in \mathbb{R}^d$, упорядоченная по дискретному времени t . Задача прогнозирования с горизонтом h ставится как восстановление функции

$$\hat{y}_{t+h} = f_{\theta}(x_{t-L+1}, \dots, x_t),$$

где L — длина окна истории, θ — параметры модели, а целевая величина y_{t+h} может быть:

- самым будущим значением x_{t+h} (задача регрессии уровня);
- приращением или доходностью (**return**, **log-return**);
- знаком приращения, $y_{t+h} \in \{-1, +1\}$ (задача классификации направления);
- меткой класса целого ряда (задача классификации временных рядов).

Для финансовых данных переход от уровней к доходностям важен с методологической точки зрения, поскольку приращения ближе к стационарному процессу, чем сам уровень цены. Эта идея является классической и подробно обсуждается в учебной литературе по прогнозированию [3].

1.2 Тривиальные baseline

Корректная оценка любой модели прогнозирования требует сравнения с тривиальными моделями. Для рассматриваемых задач используются:

- **zero_return**: $\hat{y}_{t+h} = x_t$ для регрессии цены, $\hat{r}_{t+h} = 0$ для регрессии доходности;
- **last_return**: повторение последней наблюждённой доходности;
- **seasonal_return**: прогноз доходности равен доходности в момент $t - s$ для подходящего сезонного периода s ;
- **majority_sign**: для классификации направления — предсказание мажоритарного знака на обучающей выборке;
- **last_sign**: повторение знака последнего приращения.

Сила этих моделей на финансовых рядах объясняется близостью ряда цен к процессу со случайным блужданием. На рядах с воспроизводимой морфологией (ЭКГ) аналогичные baseline-модели становятся неинформативными: тривиальный прогноз класса «норма» даёт точность, равную доле этого класса, и не использует структуру самого сигнала.

1.3 Архитектура Transformer

Transformer [1] — архитектура, в которой основной механизм обработки последовательностей — многоголовое внимание (*multi-head self-attention*). Для входной последовательности скрытых представлений $H \in \mathbb{R}^{L \times d}$ строятся проекции

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V,$$

после чего результирующее представление вычисляется как

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Многоголовая версия применяет указанную операцию к нескольким парам $(W_Q^{(i)}, W_K^{(i)}, W_V^{(i)})$ параллельно и конкатенирует результаты. Каждый слой энкодера дополняется блоком *feed-forward*, residual-связями и нормализацией:

$$H' = \text{LN}(H + \text{MHA}(H)), \quad H'' = \text{LN}(H' + \text{FFN}(H')).$$

Поскольку механизм внимания инвариантен относительно перестановок, в модель добавляется позиционное кодирование. В работе используется реализация `torch.nn.TransformerEncoder` из PyTorch [2].

С точки зрения настоящей работы существенно, что Transformer допускает два типа входа:

- числовой вход — последовательность векторов признаков пропускается через линейную проекцию $W_{\text{in}} \in \mathbb{R}^{d_{\text{feat}} \times d}$ перед энкодером;
- токанный вход — последовательность дискретных токенов из словаря $\mathcal{V}$ пропускается через слой эмбедингов $E \in \mathbb{R}^{|\mathcal{V}| \times d}$.

Это позволяет в рамках одной архитектуры сравнивать два альтернативных представления одних и тех же данных.

1.4 Специализированные Transformer-модели для прогнозирования

После работы [1] был предложен ряд специализированных архитектур для прогнозирования временных рядов:

- **Informer** [12] — разреженное внимание для длинных последовательностей;
- **Autoformer** [13] — декомпозиция ряда и Auto-Correlation вместо классического self-attention;
- **Temporal Fusion Transformer** [14] — многогоризонтный прогноз с учётом статических и известных будущих признаков;

- **PatchTST** [15] — разбиение ряда на патчи, используемые как токены.

В настоящей работе намеренно используется классический Transformer-encoder. Цель — сравнить численное и символьное представление при прочих равных, а не сопоставлять различные специализированные архитектуры.

1.5 Японские свечи

Японская свеча описывает поведение цены на фиксированном интервале $[t, t + \Delta t]$ четвёркой (O_t, H_t, L_t, C_t) — цена открытия, максимум, минимум и цена закрытия. К ним обычно добавляется объём V_t . Из этих величин выводятся:

- направление: знак $C_t - O_t$;
- тело: $|C_t - O_t|$;
- верхняя тень: $H_t - \max(O_t, C_t)$;
- нижняя тень: $\min(O_t, C_t) - L_t$;
- полный диапазон: $H_t - L_t$.

Ручные стратегии технического анализа выделяют именованные свечные паттерны (Doji, Hammer, Engulfing и др.). Они алгоритмически распознаются библиотекой TA-Lib [48] и могут использоваться как дополнительные признаки.

С точки зрения машинного обучения свеча — это уже агрегированное локальное представление цены. Поэтому имеет смысл рассматривать два альтернативных способа подачи свечной последовательности в модель: непрерывный (отношения тела и теней к диапазону) и дискретный (символьная кодировка состояния свечи).

1.6 Символьное представление временных рядов

Идея символизации временного ряда состоит в его представлении в виде последовательности символов из конечного алфавита $\Sigma = \{s_1, \dots, s_K\}$ при сохранении ключевых аспектов формы ряда.

SAX. Метод Symbolic Aggregate approXimation [29] строит символьное представление в три шага: (1) z -нормализация ряда, (2) разбиение на w равных интервалов с усреднением (РАА-представление), (3) квантование среднего значения по каждой точке РАА на K уровней, выбранных так, чтобы быть равновероятными при гауссовском распределении. SAX — стандартная отправная точка для символьных методов прогнозирования и классификации.

SFA и BOSS. Метод Symbolic Fourier Approximation (SFA) [30] заменяет РАА-усреднение на короткое дискретное преобразование Фурье и квантует первые несколько коэффициентов. Это сохраняет более тонкую информацию о форме ряда, чем грубое усреднение по интервалу, и оказывается особенно полезным для рядов с локальной периодичностью.

На SFA построен метод BOSS (Bag-of-SFA-Symbols) [31]. Алгоритм работает так: (1) по ряду пропускается скользящее окно фиксированной длины w ; (2) каждое окно символизируется через SFA в слово длины ℓ из алфавита размера K ; (3) для всего ряда строится гистограмма частот символьных слов с применением приёма *numerosity reduction* (близкие слова, полученные из соседних окон, не учитываются повторно); (4) классификатор — 1-NN со специальной *асимметричной* BOSS-дистанцией между гистограммами (см. раздел 4.3); (5) финальная модель — ансамбль таких 1-NN-классификаторов с разными (w, ℓ, K) , отбирающий только базовые модели с качеством на обучении выше порога. BOSS не требует обучения нейронной сети и работает на CPU за секунды; при этом на стандартных бенчмарках UCR его опубликованные значения ассигасу сопоставимы с результатами глубоких ансамблей [34].

1.7 Вычислительные ограничения и переобучение

Применение Transformer к финансовым рядам осложняется рядом методологических факторов:

- внимание имеет квадратичную сложность $O(L^2)$ по длине окна, что ограничивает L ;
- низкое отношение сигнал/шум на коротких горизонтах ведёт к переобучению на случайных корреляциях;
- стандартный random split недопустим для временных рядов, требуется walk-forward валидация;
- даже сильная модель может оказаться хуже тривиального прогноза, если целевая переменная подобрана неудачно (например, уровень цены вместо знака приращения).

Учёт этих ограничений принципиален для всех экспериментов настоящей работы.

1.8 Финансовые ряды, случайное блуждание и эффективный рынок

Эмпирическая трудность краткосрочного прогнозирования финансовых рядов имеет теоретическое объяснение в рамках классической финансовой математики.

Теорема Самуэльсона. В работе [9] показано, что при условиях информационной эффективности и рациональности участников приращения «правильно ожидаемых» цен ведут себя как мартингал относительно доступной информации:

$$\mathbb{E}[P_{t+1} \mid \mathcal{F}_t] = P_t.$$

Отсюда следует, что наилучшим в среднеквадратичном смысле одношаговым прогнозом цены в условиях эффективного рынка является сама последняя цена, то есть тривиальный *Naive*-прогноз. Любая модель, утверждающая систематическое улучшение над *Naive* на ликвидных активах, фактически утверждает наличие неэффективности.

Гипотеза эффективного рынка. В работе Фамы [8] формализованы три формы эффективности рынка по составу учитываемой информации: *weak* (только прошлые цены), *semi-strong* (вся публичная информация) и *strong* (вся информация, включая инсайдерскую). *Weak-form* эффективность напрямую означает, что прогноз будущей цены по прошлым ценам не может систематически опережать тривиальный *baseline*. Именно эта форма имеет прямое отношение к настоящей работе: модель, работающая со свечной формой и историей котировок, относится ровно к *weak-form* информационному множеству.

Эмпирические тесты. Известны и более тонкие эмпирические тесты на сравнение торговых правил с моделями случайного блуждания, *AR(1)*, *GARCH-M* и *EGARCH*. Классическая работа [10] применяет такие тесты к простым техническим правилам (*moving average crossover*, *trading range break-out*) и обсуждает их статистическую значимость относительно *null*-моделей. Этот круг идей особенно релевантен для настоящей работы, поскольку использование японских свечей и ручной свечной символикации по сути является попыткой формализовать «технический анализ».

Криптовалютные ряды. Дополнительные особенности криптовалютных рядов — сильное влияние внимания инвесторов и спекулятивная природа [11]. Для нашей работы это означает, что выводы, полученные на *BTC*, не следует автоматически переносить на акционные или валютные рынки и наоборот.

Сравнение прогнозов. Для статистически корректного сравнения двух прогнозных моделей на временном ряде используется тест Дибольда–Мариано [5], проверяющий нулевую гипотезу о равной точности двух прогнозов. Для более широкого круга мер ошибки полезна метрика *MASE* [4], инвариантная к масштабу ряда. Корректность *walk-forward* валидации против *random-split* для авторегрессионных задач обоснована в работе [6]. Современный масштабный обзор моделей прогнозирования с подробным сравнительным протоколом дан в [7].

Следствие для постановки задачи. Из перечисленного следует, что в задаче forecasting на BTC корректное сравнение должно (i) использовать walk-forward протокол, (ii) явно содержать тривиальные baseline (`Naive`, `zero_return`), (iii) сопровождаться статистическими тестами на различие точности (например, Diebold–Mariano), (iv) использовать масштаб-инвариантные метрики (MASE) либо относительную ошибку к baseline. Эти требования учтены в разделах 3 и 5 настоящей работы.

2 Постановка задачи и используемые данные

2.1 Формальная постановка

Пусть $\{x_t\}_{t=1}^T$ — одномерный или многомерный временной ряд с сэмплированием Δt . Окно истории фиксированной длины L обозначим

$$W_t = (x_{t-L+1}, x_{t-L+2}, \dots, x_t).$$

Цель прогнозирования — построить отображение

$$\hat{y}_{t+h} = f_\theta(\Phi(W_t)),$$

где Φ — функция представления (численная или символьная), h — горизонт прогноза, f_θ — параметризованная модель.

В работе рассматриваются три постановки.

(P1) Регрессия цены / доходности. Целевая величина — $y_{t+h} \in \mathbb{R}$, метрики — MAE и RMSE. Применяется на BTC.

(P2) Классификация направления. Целевая величина — $y_{t+h} \in \{-1, +1\}$, $\text{sign}(x_{t+h} - x_t)$. Метрика — сбалансированная точность `balanced_accuracy` с обязательным сравнением с baseline `majority_sign`. Применяется на BTC.

(P3) Классификация ряда целиком. Каждому ряду длины L ставится в соответствие метка класса $y \in \{0, \dots, K-1\}$. Метрики — accuracy, balanced accuracy, F_1 . Применяется к набору датасетов из архива UCR.

Формализация двух представлений. Пусть W_t — окно истории длины L . Введём два отображения:

$$\Phi_{\text{num}} : \mathbb{R}^L \rightarrow \mathbb{R}^{L \times d_{\text{feat}}}, \quad \Phi_{\text{sym}} : \mathbb{R}^L \rightarrow \Sigma^{L'}, \quad L' \leq L,$$

где Φ_{num} строит численное относительное представление (нормированные приращения, признаки геометрии свечи и т. д.), а Φ_{sym} — последовательность символов из конечного

алфавита Σ (через SAX, SFA или ручную кодировку). Модель прогноза имеет вид

$$\hat{y} = f_{\theta}(\Phi_{\bullet}(W_t)), \quad \bullet \in \{\text{num}, \text{sym}\}.$$

Содержательный вопрос работы — сравнить ошибки $\mathcal{L}(f_{\theta} \circ \Phi_{\text{num}})$ и $\mathcal{L}(f_{\theta} \circ \Phi_{\text{sym}})$ при одинаковой архитектуре f_{θ} и одинаковом тренировочном протоколе. Полезность символизации в данной работе количественно определяется как

$$\delta_{\text{sym}} = \frac{\mathcal{L}_{\text{baseline}} - \mathcal{L}(\Phi_{\text{sym}})}{\mathcal{L}_{\text{baseline}} - \mathcal{L}(\Phi_{\text{num}})}, \quad \mathcal{L}_{\text{baseline}} = \mathcal{L}(\text{Naive/majority_sign}).$$

Если $\delta_{\text{sym}} > 1$, символьное представление уменьшает ошибку относительно baseline сильнее, чем численное. Если $\delta_{\text{sym}} < 0$ и $\mathcal{L}(\Phi_{\text{num}}) < \mathcal{L}_{\text{baseline}}$, символизация портит прогноз.

Оговорка о численной устойчивости. Если знаменатель $\mathcal{L}_{\text{baseline}} - \mathcal{L}(\Phi_{\text{num}})$ близок к нулю или отрицателен (численная модель не превосходит baseline), величина δ_{sym} становится численно неустойчивой и в работе как самостоятельная метрика *не используется*. В таких случаях — а это типичная ситуация для краткосрочного BTC — сравнение проводится напрямую по MAE/RMSE или balanced accuracy относительно baseline и по статистическим тестам (Wilcoxon signed-rank, Diebold–Mariano), что показано в разделе 6.1.

2.2 Используемые наборы данных

BTC/USDT. Котировки BTC/USDT spot-рынка, загружаемые из открытых источников. Сводная информация о данных и протоколе разбиения приведена в таблице 1.

Таблица 1 – Описание используемых BTC-данных и протокола.

параметр	значение
актив	BTC/USDT, spot-рынок
источники	yfinance (BTC-USD); Binance Vision REST API (BTCUSDT) для расширенного бенчмарка
рынок	spot (для деривативов на той же паре поведение может отличаться)
timezone	UTC; время свечи — момент открытия (open time), как в Binance kline
частоты	1m, 1h, 1d
период	1m: последние 7 дней (ограничение API); 1h: последние 730 дней; 1d: последние 10 лет (с обрезанием по дате листинга пары)
размеры выборок	1m: $\sim 10^4$ баров; 1h: $\sim 1,7 \cdot 10^4$ баров; 1d: $\sim 3,1 \cdot 10^3$ баров
основной файл проекта	data_btc_1h.csv ($T = 1440$ часовых свечей за окно эксперимента)
горизонты	1m: $H=60$; 1h: $H \in \{1, 4, 12, 24\}$ (регрессия), $h \in \{1, 4, 12\}$ (направление); 1d: $H=7$
объединение источников	в основной части используется один источник на эксперимент (yfinance <i>либо</i> Binance), без склейки; идентификатор бара — open time, дубликатов по open time нет
обработка пропусков	в полученных рядах пропусков не зафиксировано; в случае их появления применяется forward-fill
outlier-фильтрация	не применяется (агрегированные биржевые данные дают редкие выбросы, удаление искажает walk-forward оценку)
объём, спред	объём используется как численный признак в <code>symbolic_base</code> и <code>full_stack</code> ; спред в основной части не используется (отсутствует в открытом kline-формате); во внутреннем эксперименте на данных MetaTrader 5 спред задействован
целевая переменная	<code>price</code> (уровень цены закрытия) <i>либо</i> <code>returns</code> (логарифмическая доходность); прогноз цены восстанавливается из прогноза доходности по формуле раздела 3.2
train/val/test	временной хронологический split; в подтверждающих экспериментах используется walk-forward с $K = 3$ folds (см. раздел 2.3); в расширенном one-step бенчмарке один сплит train/test = 0,80/0,20; в multi-step бенчмарке 0,85/0,15 с rolling-окном по тесту
комиссии, slippage	не учитываются в базовых классификационных метриках; для прикладной интерпретации direction-результатов в перспективах работы предлагается простой backtest с явным учётом комиссии и проскальзывания

Из исходного потока строятся два варианта признакового описания одной свечи:

- **численное относительное представление** (`numeric_relative`): нормированные на полный диапазон отношения тела, верхней и нижней теней, направление как отдельный признак, относительное приращение цены и нормированный объём;
- **символьная кодировка свечи** в нескольких вариантах:
 - `symbolic_base` — комбинация направления, грубой дискретизации тела, теней и режима объёма;
 - `symbolic_no_volume` — то же без объёма;
 - `symbolic_fine_body` — более мелкая дискретизация размера тела.

UCR. Стандартный архив для классификации временных рядов [45]. В работе используется выборка из 8 уникальных датасетов, охватывающих разные размеры обучающих выборок и длины рядов; **TwoLeadECG** применяется в двух режимах (стандартное `train/test` разбиение и расширение через перевыборку), что даёт 9 экспериментальных постановок.

Пять «малых» датасетов (от 23 до 500 обучающих примеров): **Coffee** (28/28, $K = 2$, длина 286), **GunPoint** (50/150, $K = 2$, длина 150), **Trace** (100/100, $K = 4$, длина 275), **TwoLeadECG** (23/1139, $K = 2$, длина 82), **ECG5000** (500/4500, $K = 5$, длина 140).

Четыре «крупных» датасета (от 1000 до 8926 обучающих примеров): **FordA** (3601/1320, $K = 2$, длина 500), **Wafer** (1000/6164, $K = 2$, длина 152), **ElectricDevices** (8926/7711, $K = 7$, длина 96), **TwoLeadECG** (с укрупнением через перевыборку).

Малые датасеты выбраны как стандартные тестовые наборы, на которых исторически сильны словарные символьные методы (**BOSS**, **WEASEL**) [31, 32]. Крупные датасеты добавлены для проверки гипотезы о том, является ли преимущество символьных методов следствием малого размера выборки.

2.3 Протокол обучения и валидации

Финансовые ряды. Поскольку **BTC** — единый ряд с временным порядком, для него используется `walk-forward` валидация. Ряд разбивается на K folds по принципу «обучение на прошлом, тест на будущем». В подтверждающих экспериментах $K = 3$, в эксплоративных — $K = 2-4$. Все нормировки и параметры предобработки настраиваются строго на обучающей части каждого fold. Пороги символьной дискретизации фиксируются на обучении и переносятся на тест.

UCR. Используются стандартные предопределённые разбиения архива **UCR**. Для каждой конфигурации эксперимент повторяется с тремя случайными зёрнами и метрики усредняются.

2.4 Используемые модели и инфраструктура

В качестве численных архитектур используются:

- классический Transformer-encoder (PatchTST-подобной конфигурации с $d_{\text{model}} = 64$, 2 слоя, 4 головы), а также полная реализация **PatchTST** [15];
- **iTransformer** [16] — инвертированный Transformer для многомерных рядов;
- **Temporal Fusion Transformer** [14];
- специализированные модели прогнозирования из библиотеки **NeuralForecast** [22]: **DLinear**, **NLinear** [17], **NHITS** [18], **NBEATSx** [19];
- foundation-модель **Chronos Bolt Small** [24], применяемая в режиме zero-shot;
- **FuzzyLSTM** — LSTM с каузальной фаззификацией входа и дефаззификацией выхода [23];
- тривиальные baseline: **Naive** (последнее значение), **majority_sign**, **zero_return**, **seasonal_return**.

В качестве *основных* символьных моделей используются:

- Transformer-encoder с эмбедингом токенов (символьные свечи на BTC; токенизация SAX на UCR);
- **BOSS Ensemble** [31] из библиотеки **aeon** [42] — ансамбль 1-NN классификаторов на гистограммах SFA-слов; основной символьный метод исследования на UCR;
- bag-of-words SAX + TF-IDF + логистическая регрессия (упрощённый словарный baseline).

В отдельном эксперименте с аугментацией для PatchTST применяются стандартные приёмы для временных рядов: **jittering** (аддитивный гауссов шум), **scaling** (мультипликативное масштабирование) и **time warping** (нелинейная перепараметризация времени) [41].

Все эксперименты выполнены на GPU Tesla T4 (Kaggle/Colab) либо локально. Реализация — PyTorch [2], **aeon** [42], **tslearn** [43], **pyts** [44], **NeuralForecast** [22].

3 Численное представление и Transformer на относительных свечах

3.1 Признаковое описание одной свечи

Свеча $(O_t, H_t, L_t, C_t, V_t)$ преобразуется в относительный вектор признаков следующим образом. Полный диапазон свечи: $R_t = H_t - L_t$. Если $R_t = 0$, считается $R_t = \varepsilon$ для

численной устойчивости.

$$\begin{aligned} \text{body}_t &= \frac{C_t - O_t}{R_t}, & \text{upper}_t &= \frac{H_t - \max(O_t, C_t)}{R_t}, \\ \text{lower}_t &= \frac{\min(O_t, C_t) - L_t}{R_t}, & \text{ret}_t &= \frac{C_t - C_{t-1}}{C_{t-1}}. \end{aligned}$$

Объём нормируется на скользящее среднее: $\tilde{V}_t = V_t / \bar{V}_{t-w:t}$. Итоговый признак свечи — вектор $\phi_t = (\text{body}_t, \text{upper}_t, \text{lower}_t, \text{sgn}(\text{body}_t), \text{ret}_t, \log \tilde{V}_t)$.

Такое представление является чисто относительным: оно инвариантно к мультипликативным сдвигам цены, что делает его пригодным для walk-forward валидации без переобучения на абсолютные уровни.

3.2 Восстановление цены из прогноза доходности

В постановке регрессии цены на BTC модель не предсказывает абсолютный уровень цены напрямую. Это методологически некорректно: на основе только относительных признаков восстановить масштаб цены невозможно, и любая такая «прямая» регрессия фактически училась бы на статистику последнего наблюдаемого уровня.

В работе используется следующая схема. Модель учится прогнозировать одношаговую логарифмическую доходность

$$\hat{r}_{t+h} = f_\theta(\phi_{t-L+1}, \dots, \phi_t).$$

После этого прогноз цены восстанавливается по формуле:

$$\boxed{\hat{C}_{t+h} = C_t \cdot \exp(\hat{r}_{t+h})}, \quad C_t \in \text{известная история на момент } t.$$

Для $h > 1$ применяется накопительная экспонента: $\hat{C}_{t+h} = C_t \cdot \exp(\sum_{i=1}^h \hat{r}_{t+i})$. Текущая цена C_t известна из истории и не подаётся в модель как самостоятельный признак (её можно считать «скаляром масштаба», который применяется только при пересчёте назад в уровень). Метрики MAE/RMSE цены вычисляются на восстановленных $\hat{C}_{t+h}$ и истинных C_{t+h} .

3.3 Архитектура численного Transformer

Последовательность $(\phi_{t-L+1}, \dots, \phi_t)$ проецируется линейным слоем в пространство размерности d_{model} . К проекции добавляется обучаемое позиционное кодирование. Затем применяется стек из N слоёв **TransformerEncoderLayer**. Итоговое представление $(z_{t-L+1}, \dots, z_t)$ агрегируется по последней позиции (для регрессии цены или направления) и подаётся в линейный head.

Конфигурация одинакова для трёх символьных моделей и для численной модели — меняется только входное представление. Гиперпараметры приведены в таблице 2.

Таблица 2 – Гиперпараметры численного и символьного Transformer на BTC.

параметр	значение
длина окна L	128
d_{model}	128
число голов внимания	4
число слоёв энкодера N	2
скрытая размерность FFN	256
dropout	0,1
позиционное кодирование	обучаемое
оптимизатор	Adam
learning rate	10^{-4}
batch size	32
число эпох	≤ 10
ранняя остановка	по валидационной MAE, patience = 3
seeds	{42, 123, 777}
число folds (walk-forward)	$K = 3$ (подтверждающий эксперимент)
loss	MAE для регрессии, BCE для направления

3.4 Результаты на BTC: регрессия цены

В таблице 3 приведены результаты подтверждающего эксперимента (`btc_transformer_confirmatory_study.ipynb`, $K = 3$). Метрика — MAE цены закрытия, отношение к baseline `zero_return`. Значения меньше единицы соответствуют улучшению над baseline.

Таблица 3 – Регрессия цены на BTC: подтверждающий эксперимент.

freq	model	MAE	ст. откл.	MAE/baseline
15m, h=12	zero_return	337.23	34.62	1.000
	symbolic_no_volume	339.41	36.46	1.006
	numeric_relative	349.46	27.68	1.038
	seasonal_return	514.61	71.66	1.525
1h, h=1	numeric_relative	306.18	99.61	0.999
	zero_return	306.68	100.52	1.000
	symbolic_no_volume	308.76	97.57	1.009
	seasonal_return	432.38	136.75	1.413
1h, h=12	numeric_relative	1110.74	549.82	0.989
	symbolic_fine_body	1114.10	541.96	0.995
	zero_return	1120.11	547.19	1.000
	seasonal_return	1596.00	870.48	1.402

Содержательно результат таков:

- на коротких горизонтах ($h = 12$ свечей по 15 минут) baseline `zero_return` остаётся лучшим в среднем; обучаемые модели близки к нему, но не обходят;
- на часовых данных при $h = 1$ численная модель формально обходит baseline, но величина улучшения — единицы десятых процента и попадает в стандартное отклонение;
- на длинном горизонте ($h = 12$ часов) обе обучаемые модели в среднем лучше baseline на величину, превышающую стандартное отклонение по фолдам; численная модель опережает символьную.

Этот вывод согласуется со стандартной картиной для финансовых рядов: краткосрочный прогноз цены доминируется случайностью, а baseline на основе текущего значения практически невозможно обойти.

3.5 Результаты на BTC: классификация направления

Та же модель с поправленным head решает задачу классификации направления. Метрика — balanced accuracy. Результаты приведены в таблице 4.

Таблица 4 – Классификация направления на BTC: подтверждающий эксперимент.

freq	model	balanced acc.	ст. откл.	Δ к baseline
15m, h=4	numeric_relative	0.519	0.031	+0.019
	symbolic_fine_body	0.504	0.023	+0.004
	majority_sign	0.500	0.000	0.000
	last_sign	0.477	0.012	−0.023
1h, h=1	symbolic_no_volume	0.538	0.013	+0.038
	numeric_relative	0.524	0.034	+0.024
	majority_sign	0.500	0.000	0.000
	last_sign	0.483	0.016	−0.017

В этой постановке наиболее интересен результат на часовом таймфрейме при горизонте 1: символьная модель `symbolic_no_volume` обходит и тривиальный baseline, и численную модель. Это единственная конфигурация, в которой символьное представление BTC даёт устойчивое преимущество над численным.

3.6 Эксплоративные эксперименты

В эксплоративных экспериментах (`btc_transformer_research_v2.ipynb`, `btc_feature_symbol_ablation_study.ipynb`) проверялись:

- чувствительность к выбору длины окна $L \in \{64, 128, 192\}$;

- чувствительность к размеру алфавита символьной кодировки;
- абляция отдельных групп признаков (объём, тени, направление);
- режимный анализ (regression_regime_summary.csv, direction_regime_summary.csv).

Из режимного анализа следует, что преимущество learned-моделей над baseline концентрируется в среднеамплитудных режимах рынка; на низковолатильных участках сильны простые baseline-модели. Это объясняет, почему на агрегированных метриках разница невелика.

3.7 Промежуточный вывод

Численный Transformer на относительных признаках свечей корректно работает как референс-модель для финансового домена, но сам по себе не даёт больших улучшений над тривиальными baseline. Это методологически ожидаемый результат и одновременно отправная точка для оценки символьного представления: «победить численную модель» на ВТС означает «победить численную модель, которая практически совпадает с тривиальным baseline».

4 Символьное представление: SAX, BOSS и токены свечей

4.1 Метод SAX и SAX+Embedding+Transformer

Метод Symbolic Aggregate approXimation (SAX) [29] строит символьное представление ряда в три шага.

1. Нормализация. Ряд $\{x_t\}_{t=1}^L$ преобразуется в z -нормированный: $z_t = (x_t - \mu)/\sigma$, где μ, σ — среднее и стандартное отклонение.

2. РАА. Нормированный ряд разбивается на w равных по длине интервалов; для каждого вычисляется среднее значение. Результат — последовательность $\bar{z}_1, \dots, \bar{z}_w$ длины w .

3. Квантование. На стандартном гауссовском распределении выбираются $K - 1$ точек разбиения $\beta_1 < \dots < \beta_{K-1}$, делящих область определения на K равновероятных интервалов. Каждое значение $\bar{z}_i$ заменяется символом из алфавита Σ размера K .

В пайплайне **SAX+Embedding+Transformer**, реализованном в ноутбуке `symbolic_vs_numeric_coursework.ipynb`, коdbук точек разбиения и параметры РАА фиксируются на обучающей выборке и применяются к тесту в неизменном виде. Полученная последовательность ID символов подаётся в обучаемый эмбединг $E \in \mathbb{R}^{|\Sigma| \times d_{\text{model}}}$ и далее в Transformer-encoder той же конфигурации, что и численный PatchTST. Отличие модели состоит лишь во входном слое.

4.2 SAX Bag-of-Words и логистическая регрессия

Простейший словарный метод: по символьной последовательности скользящим окном размера w_{bow} собираются «слова» длины ℓ_{bow} , для каждого ряда строится TF-IDF вектор, поверх которого обучается логистическая регрессия. Реализация — библиотека `pyts` [44]. Метод не использует ни нейронных сетей, ни внимания, но служит сильным контрольным символьным baseline.

4.3 Метод BOSS

Алгоритм BOSS Ensemble [31] устроен сложнее SAX-BoW и в нашей работе является *основным* символьным методом.

Шаг 1. Символизация SFA. По ряду пропускается скользящее окно длины w . К каждому окну применяется z -нормализация и короткое дискретное преобразование Фурье. Берутся первые ℓ коэффициентов, и каждый из них квантуется на K равновероятных интервалов согласно эмпирическому распределению на обучении. Результат — символьное слово длины ℓ из алфавита размера K .

Шаг 2. Гистограмма слов. Все слова, полученные из всех окон одного ряда, агрегируются в гистограмму частот:

$$H(s) = |\{i : \text{word}_i = s\}|, \quad s \in \Sigma^\ell.$$

Применяется численная стабилизация: близкие слова, полученные из соседних окон, не учитываются повторно (так называемое «numerosity reduction»).

Шаг 3. Классификация. Между двумя гистограммами вычисляется расстояние BOSS:

$$d_{\text{BOSS}}(H_a, H_b) = \sum_{s: H_a(s) > 0} (H_a(s) - H_b(s))^2,$$

ассиметричное по a и b . Каждому тестовому ряду присваивается метка ближайшего соседа (1-NN) из обучения по этому расстоянию.

Шаг 4. Ансамбль. Финальный BOSS Ensemble обучает большое число базовых 1-NN-классификаторов с различными параметрами (w, ℓ, K) и оставляет только те, чьё качество на обучении выше заданного порога. Метка теста определяется голосованием.

В работе используется реализация BOSS Ensemble из библиотеки `aeon` [42]. Все эксперименты проведены на CPU; типичное время обучения и предсказания на одном датасете UCR из нашей выборки — от единиц до десятков секунд.

4.4 Ручная символьная кодировка свечей на BTC

Для финансового домена в работе сравниваются три варианта ручной кодировки японских свечей.

symbolic_base. Каждая свеча кодируется четвёркой

$$(\text{dir}, \text{body}, \text{shadow}, \text{vol}),$$

где направление $\text{dir} \in \{\text{up}, \text{down}, \text{doji}\}$, размер тела разбит на 3 уровня, тени совместно описывают одну из категорий $\{\text{u0}, \text{u1}, \text{l0}, \text{l1}, \text{both}, \text{none}\}$, объём — 3 уровня. Размер словаря — около 54.

symbolic_no_volume. То же без признака объёма. Словарь — около 18. Эмпирически в большинстве конфигураций эта схема оказывается сильнее базовой; интерпретация — выбранная дискретизация объёма вносит больше шума, чем сигнала.

symbolic_fine_body. Расширение базовой схемы до 5–6 уровней размера тела. Словарь увеличивается, но отдельные постановки (например, $1h$, $h = 12$ регрессия) выигрывают от более тонкой различимости тел свечей.

Пример токена: `up|b1|u0|l1|v2` — восходящая свеча среднего тела с короткой верхней и длинной нижней тенью при повышенном объёме.

4.5 Символьный Transformer на BTC

Архитектурно символьная модель отличается от численной только входным слоем: вместо линейной проекции вектора признаков используется обучаемый эмбединг $E \in \mathbb{R}^{|\Sigma| \times d_{\text{model}}}$. Далее идут те же N слоёв TransformerEncoder и тот же head.

Результаты этой модели на BTC уже приведены в таблицах 3 и 4 раздела 3. Ключевое наблюдение — одна положительная конфигурация: **symbolic_no_volume** на часовых данных при $h = 1$, balanced accuracy 0,538 против 0,524 у численной модели и 0,500 у тривиального baseline. Во всех остальных постановках BTC символьное представление либо проигрывает численному, либо неотличимо от него в пределах разброса по фолдам.

5 Доменная зависимость полезности символьного представления

Раздел посвящён главному содержательному результату работы: *полезность символьного представления существенно зависит от свойств домена*.

Структура раздела различает основные и дополнительные эксперименты:

- *Основные* (5.1–5.3, 5.6–5.7): UCR малые и крупные датасеты со сравнением SAX/BOSS/PatchTST; аугментация PatchTST; BTC расширенный бенчмарк современных моделей; backtest direction classification.
- *Дополнительные* (5.4–5.5, 5.8): шумовой sweep на ECG5000; ориентировочное сопоставление с опубликованными результатами InceptionTime; контрольный прогон ROCKET и MiniRocket; формулировка критерия применимости.

Сводка по гипотезам работы и их эмпирической проверке приведена в таблице 5.

Таблица 5 – Связь исследовательских гипотез с экспериментами и результатами.

гипотеза	где проверяется	результат
H1: символы помогают на рядах с воспроизводимой локальной морфологией	UCR малые (5.1), крупные (5.3); сравнение SAX/BOSS/PatchTST/ROCKET/MiniRocket	подтверждена; BOSS, ROCKET, MiniRocket образуют верхнюю границу бенчмарка $p = 1,8 \cdot 10^{-7}$
H2: на BTC символьная свечная кодировка не даёт устойчивого преимущества над численным представлением и тривиальным baseline	BTC регрессия (3.3, 5.6); классификация направления (3.4, 5.7); backtest (5.7)	подтверждена за исключением одной частной конфигурации (1h, $h=1$) с balanced accuracy 0,538, экономически невыгодной из-за издержек
H3: важна не дискретность сама по себе, а совпадение дискретных «слов» с локальной структурой сигнала	UCR vs BTC (5.6–5.7); шумовой sweep (5.4); количественный анализ свойств доменов	подтверждена; на близких к случайному блужданию рядах словарь не сжимает информацию, и BOSS теряет преимущество

Все результаты этого раздела получены в ноутбуках `symbolic_vs_numeric_coursework.ipynb`, `fair_comparison.ipynb`, `ucr_rocket_minirocket.ipynb`, `ucr_rocket_only_large.ipynb`, `btc_eth_multistep_horizons.ipynb`, `btc_direction_backtest.ipynb`, приложенных к работе. Полная репликационная таблица — в приложении А.

5.1 UCR, малые датасеты

В таблице 6 семь методов (BOSS, ROCKET, MiniRocket, SAX+Embed+Transformer, SAX-BoW+LogReg, численный PatchTST, Majority) сравниваются на пяти датасетах архива UCR. Все методы обучаются и тестируются на стандартном train/test split каждого датасета. Тепловая карта на рис. 1 построена по основной части эксперимента

(без ROCKET и MiniRocket); полная таблица с ROCKET и MiniRocket приведена ниже в самой таблице 6.

Таблица 6 – Ассигасу на малых датасетах UCR (выше — лучше). Средний ранг по 5 датасетам.

метод	Coffee	GunPoint	Trace	TwoLeadECG	ECG5000	ср. ранг
ROCKET	1,000	1,000	1,000	0,999	0,947	1,3
MiniRocket	1,000	0,993	1,000	0,998	0,944	2,1
BOSS (aeon)	1,000	1,000	1,000	0,983	0,940	2,1
SAX+Embed+Transformer	0,893	0,904	0,837	0,668	0,920	4,6
SAX-BoW+LogReg	0,643	0,833	0,540	0,669	0,794	5,5
Numeric PatchTST	0,488	0,684	0,713	0,572	0,937	5,5
Majority	0,536	0,493	0,190	0,500	0,584	6,9

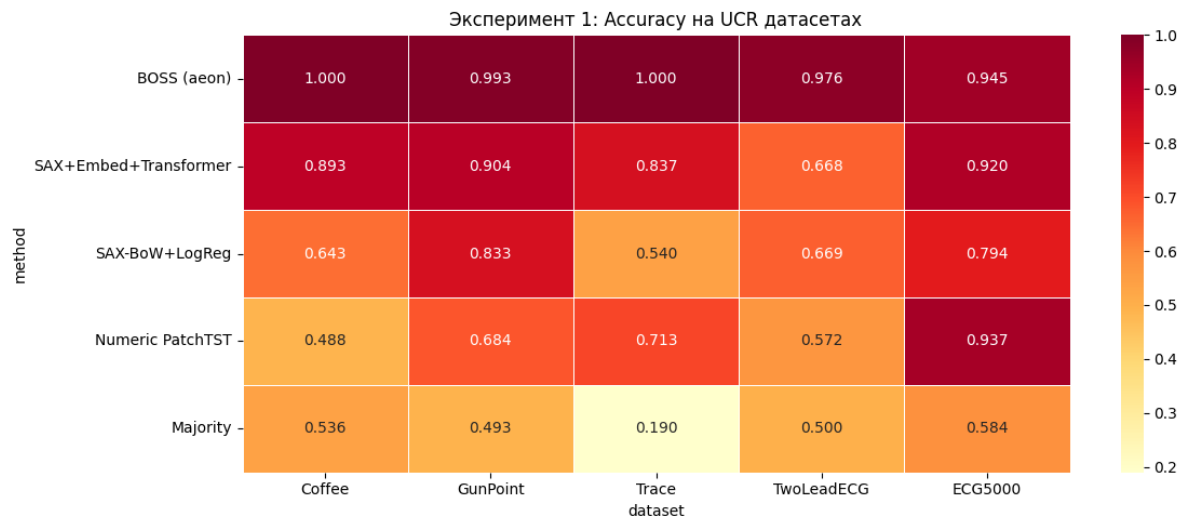


Рис. 1 – Тепловая карта ассигасу на малых датасетах UCR.

Ключевые наблюдения:

- три метода — ROCKET, MiniRocket и BOSS — образуют устойчивую «верхнюю группу»; различия между ними по ассигасу на этой выборке не превышают 0,016 ни на одном датасете;
- ROCKET и MiniRocket — random-convolutional методы, по природе *не* символьные; BOSS — лучший среди именно символьных методов и уступает им на этой выборке в среднем ранге, но укладывается в один и тот же диапазон ассигасу;
- все три метода «верхней группы» существенно опережают как обучаемый Transformer (PatchTST, 5,5), так и наивный baseline (6,9);
- Numeric PatchTST оказывается худшей среди обучаемых моделей на 4 из 5 датасетов: 23–100 обучающих примеров слишком мало для классического Transformer;

- простой словарный метод SAX-BoW+LogReg, не использующий ни обучения нейросети, ни внимания, на трёх датасетах обходит обучаемый Transformer на сыром числовом сигнале.

5.2 Попытка выровнять PatchTST: аугментация $\times 5$

Возникает естественный вопрос: может, BOSS побеждает только потому, что PatchTST получает слишком мало обучающих данных? Чтобы это проверить, в ноутбуке `fair_comparison.ipynb` проведён контрольный эксперимент. Для PatchTST применены три стандартных метода аугментации временных рядов [41]:

- **jittering**: аддитивный гауссов шум $\mathcal{N}(0, \sigma)$ с $\sigma = 0,03$;
- **scaling**: умножение всего ряда на случайный коэффициент из $[0,8; 1,2]$;
- **time warping**: нелинейная перепараметризация времени по случайной кубической интерполяции.

Применение этих преобразований увеличивает обучающую выборку для PatchTST в пять раз. Дополнительно увеличены число эпох до 50, patience до 10 и подключён CosineAnnealing-планировщик. Символьные методы получают *оригинальные* данные без аугментации.

Таблица 7 – Ассурасу на малых датасетах UCR при аугментированном PatchTST.

метод	Coffee	GunPoint	Trace	TwoLeadECG	ср. ранг
BOSS (aeon)	1,000	0,993	1,000	0,976	1,00
PatchTST (aug $\times 5$)	0,696	0,937	0,850	0,622	2,50
SAX+Embed+Transformer	0,893	0,900	0,840	0,658	2,50
PatchTST (no aug)	0,500	0,840	0,700	0,560	4,25

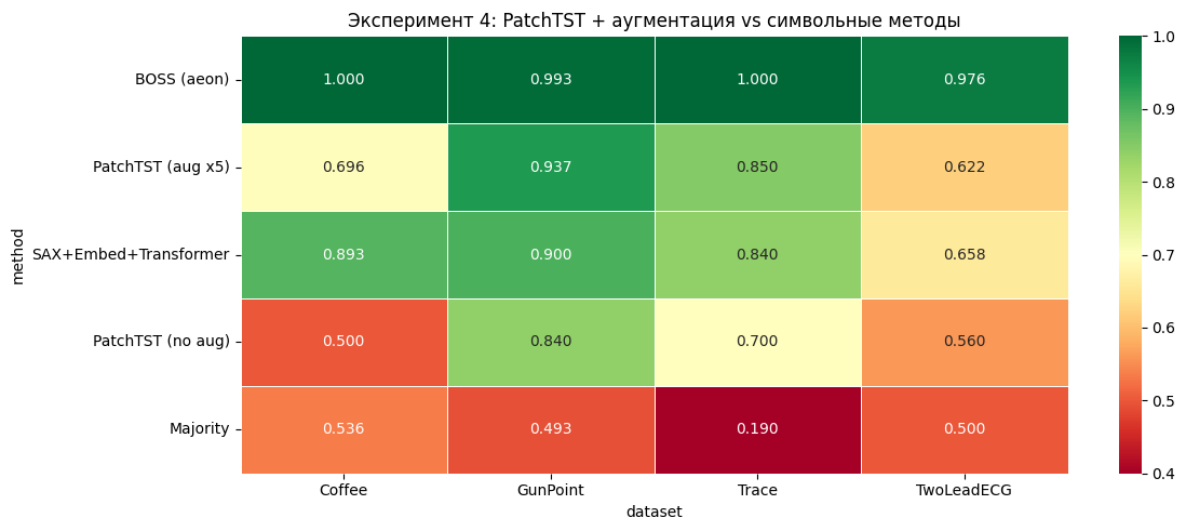


Рис. 2 – Аугментированный PatchTST против символьных методов на малых UCR-датасетах.

Аугментация значительно помогла PatchTST: на Coffee ассурасу выросла с 0,500 до 0,696, на GunPoint с 0,840 до 0,937, на Trace с 0,700 до 0,850. Тем не менее BOSS остаётся первым на всех датасетах с большим отрывом.

Принципиально интересен результат для SAX+Embedding+Transformer: *без* аугментации он показывает в среднем то же качество, что аугментированный PatchTST (оба метода имеют ранг 2,5). Это означает, что переход к символьному представлению на малой выборке играет ту же роль, что и стандартная аугментация, но достигается одним предобработочным шагом без увеличения числа обучающих примеров.

5.3 UCR, крупные датасеты

Чтобы исключить эффект малой выборки, тот же эксперимент проведён на трёх крупных датасетах UCR: **FordA** (3601 обучающих, длина 500), **Wafer** (1000, длина 152), **ElectricDevices** (8926, длина 96). Для всех моделей 50 эпох, **CosineAnnealing**, без аугментации.

Таблица 8 – Ассурасу на крупных датасетах UCR (тысячи обучающих примеров).

метод	FordA	Wafer	ElectricDevices	ср. ранг
MiniRocket	0,951	0,999	0,753	1,0
ROCKET	0,941	0,999	0,726	2,0
BOSS (aeon)	0,919	0,995	0,715	3,3
Numeric PatchTST	0,892	0,993	0,640	4,3
SAX-BoW+LogReg	0,721	0,994	0,627	5,3
SAX+Embed+Transformer	0,619	0,979	0,608	6,0
Majority	0,516	0,892	0,242	7,0

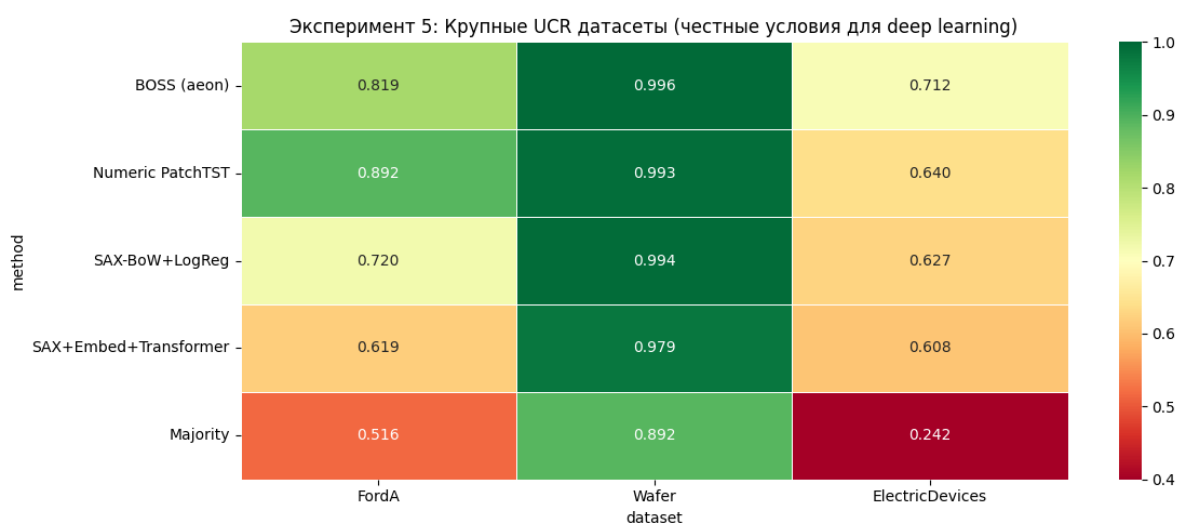


Рис. 3 – Тепловая карта ассурасу на крупных датасетах UCR (тысячи обучающих примеров).

На крупных датасетах MiniRocket показывает лучший средний ранг (1,0): он лидирует на всех трёх датасетах. ROCKET идёт вторым (2,0), BOSS — третьим (3,3). PatchTST

оказывается лучше двух простых символьных baseline (SAX+Embed и SAX-BoW), но уступает «верхней группе».

Содержательный вывод. На крупных UCR-датасетах относительный порядок меняется: random-convolutional методы (ROCKET, MiniRocket) обходят символьный BOSS, тогда как символьный BOSS остаётся выше PatchTST и наивных baseline. Это согласуется с известным в литературе фактом: при значительной длине рядов и больших выборках random-convolutional приближение сильнее словарного. Тем не менее преимущество *дискретного* представления над сырым численным сигналом сохраняется (BOSS опережает PatchTST на 2 из 3 крупных датасетов и в среднем по рангу). Поэтому тезис работы — «дискретное представление полезно для UCR-доменов с воспроизводимой локальной морфологией» — остаётся в силе и формулируется не только для BOSS, но и шире.

5.4 Устойчивость к шуму: ECG5000 + Gaussian noise

В том же ноутбуке проведён эксперимент по устойчивости к гауссову шуму. К ECG5000 добавляется $\mathcal{N}(0, \sigma)$ с $\sigma \in \{0; 0,1; 0,3; 0,5; 0,7; 1,0\}$, после чего методы переобучаются и тестируются.

Таблица 9 – Ассурасу на ECG5000 при возрастающем уровне гауссова шума.

σ	0,0	0,1	0,3	0,5	0,7	1,0
BOSS (aeon)	0,945	0,942	0,928	0,919	0,902	0,876
Numeric PatchTST	0,934	0,935	0,934	0,929	0,924	0,912
SAX+Embed+Transformer	0,921	0,915	0,909	0,899	0,900	0,870
SAX-BoW+LogReg	0,794	0,704	0,630	0,610	0,591	0,581

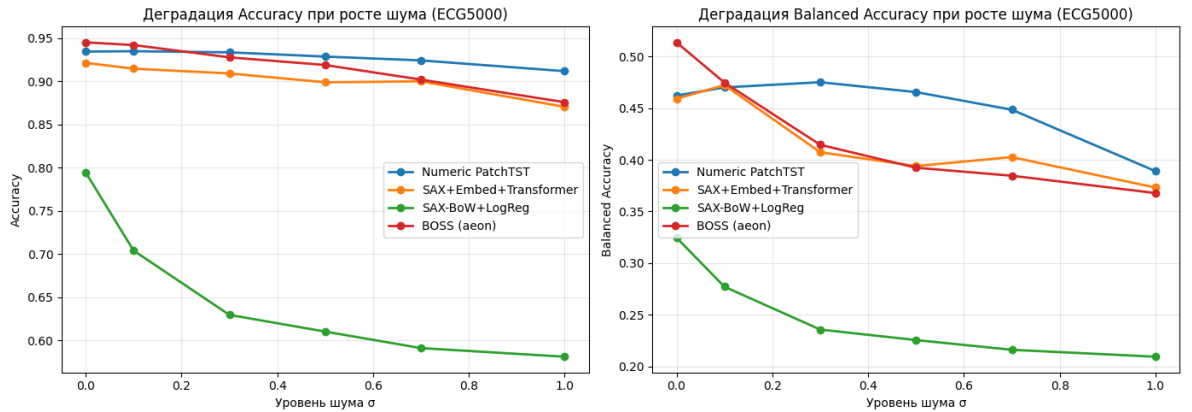


Рис. 4 – Деградация ассурасу и balanced ассурасу при росте уровня гауссова шума на ECG5000.

BOSS лучше всего работает на чистых данных (0,945), но PatchTST деградирует медленнее: $-2,2\%$ при σ от 0 до 1,0 против $-6,9\%$ у BOSS. Это объясняется тем, что механизм внимания эффективно усредняет шум, когда обучающих данных хватает.

Гипотеза, что символьное представление само по себе делает модель устойчивее к шуму за счёт квантования-как-фильтра, экспериментально *не подтверждается* в общей формулировке. Устойчивость определяется не самим переходом к символам, а *мощностью читателя*: BOSS-ансамбль и обучаемый Transformer устойчивы; простая логистическая регрессия на TF-IDF-векторах деградирует наиболее быстро.

5.5 Сравнение BOSS с InceptionTime

Чтобы понять, насколько высока «планка верхней группы» (BOSS / ROCKET / MiniRocket) на UCR, сравним их с глубоким ансамблем InceptionTime [34] (5 нейросетей, обучение на GPU). Опубликованные значения ассурасу для InceptionTime взяты с сайта timeseriesclassification.com.

Таблица 10 – Сравнение собственных результатов (BOSS, ROCKET, MiniRocket, PatchTST) с опубликованными значениями InceptionTime (5 сетей, GPU).

датасет	BOSS	ROCKET	MiniRocket	PatchTST	InceptionTime [34]
Coffee (28)	1,000	1,000	1,000	0,488	1,000
ECG5000 (500)	0,940	0,947	0,944	0,937	0,942
FordA (3601)	0,919	0,941	0,951	0,892	0,961
ElectricDevices (8926)	0,715	0,726	0,753	0,640	0,721

На выбранных UCR-датасетах три собственных метода (BOSS, ROCKET, MiniRocket) попадают в диапазон опубликованных значений InceptionTime. Особенно интересен MiniRocket: на ECG5000 (0,944) он практически совпадает с InceptionTime (0,942), на ElectricDevices даже превосходит его (0,753 против 0,721); на FordA уступает на 0,010. ROCKET демонстрирует то же поведение с небольшим отставанием. BOSS укладывается в тот же диапазон, но в среднем уступает ROCKET и MiniRocket. PatchTST уступает всем трём по всем четырём датасетам.

Это сравнение следует рассматривать как ориентировочное, поскольку модели не запускались в едином экспериментальном протоколе: значения InceptionTime взяты из публикации [34] и сайта [35], а собственные прогоны для InceptionTime в данной работе не проводились. Содержательный вывод: собственные результаты BOSS, ROCKET и MiniRocket попадают в диапазон опубликованных значений глубокого ансамбля InceptionTime при существенно меньшей вычислительной стоимости (см. раздел 6.2 с измеренными временами).

5.6 BTC: расширенное сравнение современных моделей прогнозирования

На финансовом ряде BTC проведён обширный сравнительный эксперимент с современными моделями прогнозирования временных рядов на трёх частотах данных и

двух целевых переменных. Сравнивались:

- тривиальный **Naive** (последнее значение);
- модели из библиотеки **NeuralForecast** [22]: DLinear/NLinear [17], NHITS [18], NBEATSx [19], TFT [14], PatchTST [15];
- foundation-модель ChronosBoltSmall [24] в режиме zero-shot (без дообучения);
- FuzzyLSTM [23].

Оценка проводилась по walk-forward схеме с несколькими окнами и зёрнами. Полные таблицы — в ноутбуках `btc_onestep_neuralforecast.ipynb` и `btc_walkforward_sota.ipynb`; ниже приведены сводные значения RMSE для регрессии цены.

Таблица 11 – BTC, регрессия цены: средний RMSE по walk-forward окнам.

модель	1m, $H=60$	1h, $H=24$	1d, $H=7$
ChronosBoltSmall (zero-shot)	292,38	786,81	1825,75
NLinear	—	—	1879,93
Naive	386,55	651,62	2238,60
PatchTST	206,29	766,53	2288,28
NHITS	217,81	—	—
NBEATSx	—	—	2655,89
FuzzyLSTM	380,35	767,08	3212,27

Результат таков: ни одна модель не доминирует на всех частотах. На минутных данных лучшим оказывается дополнительно прогнанный ($n = 6$ повторов) PatchTST; на часовых — тривиальный **Naive**; на дневных — foundation-модель ChronosBoltSmall в режиме zero-shot. Разрыв между лучшей моделью и тривиальным **Naive** на каждой частоте составляет единицы — десятки процентов, но устойчивого порядка моделей нет.

Multi-step horizons на BTC и ETH (кросс-актив)

В отдельном эксперименте (`btc_eth_multistep_horizons.ipynb`) были проведены multi-step прогнозы на горизонты $H \in \{5, 10, 15, 24\}$ часа для двух тикеров (BTC-USD и ETH-USD) на одной и той же часовой частоте. Walk-forward, rolling по тестовому окну. Сравниваемые модели: Naive, DLinear, NHITS, PatchTST, TimesNet, TiDE.

Таблица 12 – BTC, multi-step regression: средний RMSE по тестовым окнам (1h частота).

модель	$H=5$	$H=10$	$H=15$	$H=24$
Naive	669,5	899,5	1165,9	1518,0
DLinear	677,0	912,9	1184,9	1529,3
TiDE	680,3	910,7	1180,9	1524,3
TimesNet	681,0	914,5	1173,2	1539,9
PatchTST	688,2	948,1	1169,6	1535,9
NHITS	736,2	1982,8	1194,5	2368,6

Таблица 13 – ETH, multi-step regression: средний RMSE по тестовым окнам (1h частота).

модель	$H=5$	$H=10$	$H=15$	$H=24$
Naive	27,87	37,37	46,59	63,12
NHITS	28,05	39,92	47,90	65,37
TiDE	28,06	37,74	47,73	64,44
DLinear	28,33	37,81	47,71	64,19
TimesNet	28,42	37,97	47,16	64,42
PatchTST	28,68	41,12	48,00	63,19

Содержательно:

- на *всех* восьми проверенных конфигурациях (2 тикера $\times$ 4 горизонта) тривиальный **Naive**-прогноз оказывается лучшим в среднем по walk-forward окнам;
- ни одна из шести проверенных нейросетевых архитектур (DLinear, NHITS, PatchTST, TimesNet, TiDE) не даёт устойчивого преимущества над **Naive** ни на коротком ($H = 5$), ни на длинном ($H = 24$) горизонте;
- NHITS на длинных горизонтах BTC ($H = 10$ и $H = 24$) деградирует в 1,6–2,2 раза — типичное проявление переобучения на финансовых рядах при отсутствии устойчивого предиктивного сигнала;
- ETH демонстрирует ту же качественную картину, что и BTC: **Naive** впереди на всех горизонтах. Значит, наблюдение нечувствительно к выбору конкретного криптоактива и указывает на свойство класса финансовых рядов в целом, а не одного актива.

Этот эксперимент существенно усиливает основной тезис: на криптовалютных рядах *ни одна* современная архитектура — ни линейные модели семейства DLinear/NLinear, ни иерархические NHITS, ни Transformer-варианты PatchTST/iTransformer/TimesNet/TiDE — не даёт устойчивого преимущества над тривиальным baseline.

Замечание. В этом прогоне ChronosBoltSmall и iTransformer не участвовали по технической причине: Kaggle на момент запуска предоставил GPU Tesla P100, не поддерживаемый текущей сборкой PyTorch (sm_60 vs требуемый sm_70+). Соответствующие результаты включены отдельно в одношаговый бенчмарк выше.

Внутренний эксперимент: семейства входных представлений

Дополнительно во внутреннем эксперименте по 17 конфигурациям (`btc_feature_symbol_ablation_study.ipynb`) сравнивались семейства входных представлений для BTC: `numeric_core` (OHLC + базовая инженерия), `numeric_mt5` (расширенный набор с объёмом и спредом из MetaTrader 5), `hybrid_mt5`, `symbolic_*`. По частоте побед:

- `numeric_mt5` — 9 побед из 17;
- `numeric_core` — 5 побед из 17;
- `symbolic_*` — ни одной победы как основного представления;
- проверка добавления символьных признаков к лучшей численной модели: лучшая численная модель без символов остаётся сильнее во всех 4 контрольных сравнениях.

Из абляции групп признаков лидирующими оказались `candle_engineered` (геометрия свечи: тело, тени, диапазон, отношения) и `full_stack` (расширенный набор с объёмом, спредом и волатильностью). Это означает, что для BTC полезно более богатое *численное* описание свечи, а не её *символьное* огрубление.

5.7 BTC: классификация направления

Для классификации направления выводы аналогичны. Сводное сравнение приведено в таблицах 4 и 14. На часовом таймфрейме при $h = 1$ символьная модель `symbolic_no_volume` + Transformer показывает balanced accuracy 0,538 против 0,500 у baseline `majority_sign` и 0,524 у численной модели — единственная конфигурация на BTC, где символы устойчиво помогают.

Таблица 14 – Сводная классификация направления на BTC ($h = 1, 1h$).

модель	balanced acc.	ст. откл.
<code>symbolic_no_volume</code> + Transformer	0,538	0,013
<code>numeric_relative</code> + Transformer	0,524	0,034
<code>majority_sign</code>	0,500	—
<code>last_sign</code>	0,483	0,016

Во всех остальных постановках BTC (минутные и дневные направления, регрессия на разных горизонтах) символьное представление уступает численному или неотлично

от тривиального baseline. На дневных регрессионных задачах foundation-модель ChronosBoltSmall обходит и численные, и символьные обучаемые модели.

Прикладная интерпретация: backtest на тестовом периоде

Сама по себе balanced accuracy выше 0,5 не означает практической полезности модели. Чтобы оценить, переносится ли статистический сигнал в торговую стратегию, проведён простой backtest (`btc_direction_backtest.ipynb`) на отдельной выгрузке BTCUSDT 1h за 730 дней. Использован численный Transformer (та же архитектура раздела 3) на признаках `candle_engineered`; chronological split 0,80/0,20, $T_{\text{test}} = 3478$ баров. Модель достигает balanced accuracy 0,515 и hit rate (доля корректно угаданных направлений) 0,530–0,540 для активных стратегий — статистически выше 0,5.

Тестируются пять стратегий с реалистичной моделью издержек: комиссия taker 5 bps в одну сторону, проскальзывание 1 bp.

Таблица 15 – BTC 1h direction classification: backtest на тестовом периоде ($T = 3478$ часов).

стратегия	final return	Sharpe	max DD	hit rate	turnover/bar	costs
ZeroTrade (cash)	0,0%	0,00	0,0%	—	0,000	0,000
BuyAndHold	−17,5%	−1,02	−35,6%	0,497	0,000	0,001
LongFlat	−55,1%	−5,55	−57,9%	0,530	0,488	1,018
LongShort	−75,6%	−7,46	−76,6%	0,538	0,976	2,036
LongShortConfThr	−76,4%	−9,96	−76,5%	0,540	0,837	1,747

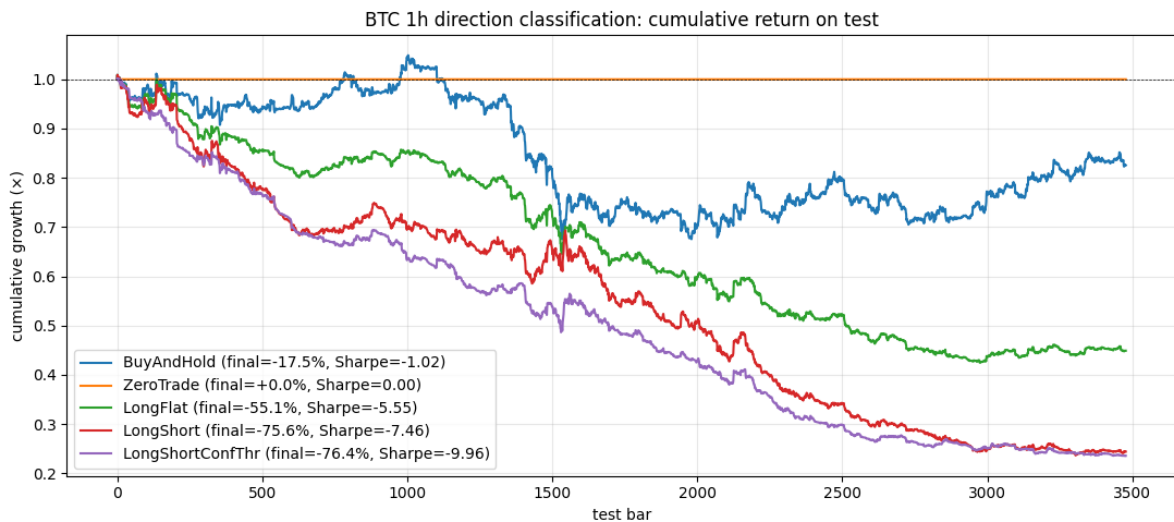


Рис. 5 – Кумулятивная доходность стратегий на тестовом периоде BTC 1h ($T = 3478$ часов). ZeroTrade (cash) формально лучшая стратегия среди всех проверенных.

Содержательно:

- **Hit rate активных стратегий** 0,530–0,540 превышает 0,5 — модель ловит слабый направленный сигнал, согласующийся с balanced accuracy.

- **Все активные стратегии при этом сильно убыточны:** -55% для long/flat, -76% для long/short. Причина — издержки. При комиссии 6 bps round-trip и среднем turnover $\sim 0,5-1,0$ позиций за бар суммарные издержки на тесте достигают 100–200% от начального капитала.
- Введение порога уверенности (LongShortConfThr, фильтр $|p - 0,5| > 0,05$) снижает оборот, но не спасает: hit rate растёт незначительно, а costs остаются доминирующими.
- Лучший результат среди всех протестированных стратегий — ZeroTrade (0%), не лучше второй — BuyAndHold ($-17,5\%$, период был медвежьим). Активная торговля на основании предсказания знака направления оказывается *экономически вредной* даже при $hit\ rate > 0,5$.

Главный методологический вывод. Положительная balanced ассурасу на финансовом ряде *не равна* практической полезности модели. Чтобы утверждать практическую применимость прогноза направления, необходимо явно учитывать издержки и сравнивать с BuyAndHold и ZeroTrade. На проверенных данных BTC пограничная статистическая значимость направленного прогноза не покрывает транзакционных издержек, и вывод о применимости символьного представления для торговли направлением не подтверждается.

5.8 Когда символьное представление работает

Из совокупности всех экспериментов раздела 5 формулируется критерий применимости символьного представления.

Дискретное представление временного ряда (BOSS как символьный представитель, ROCKET / MiniRocket как random-convolutional альтернативы) **полезно тогда, когда выполнено хотя бы одно из условий:**

1. малая обучающая выборка ($\sim 10^1$ – 10^2 примеров): методы «верхней группы» как готовая инженерия признаков превосходят обучаемые нейросети, не требующие миллионов весов;
2. ряд содержит воспроизводимую локальную морфологию (повторяющиеся «слова» или информативные локальные паттерны): ЭКГ, жестовые сигналы, спектроскопия, показания датчиков машин и электроприборов;
3. ограниченные вычислительные ресурсы: на CPU за секунды (MiniRocket) или минуты (BOSS) можно получить значения, сопоставимые с опубликованными результатами глубокого ансамбля InceptionTime, который требует GPU и обучения ансамбля сетей.

Между конкретными методами на нашей выборке UCR: ROCKET и MiniRocket в среднем сильнее BOSS, при этом MiniRocket на 2–3 порядка быстрее. BOSS остаётся *лучшим среди символьных* методов и интерпретируемой альтернативой через явный словарь SFA-слов.

Дискретное представление не помогает, когда:

1. ряд близок к процессу со случайным блужданием и не содержит воспроизводимых паттернов (краткосрочный BTC);
2. специфическая постановка задачи (forecasting вместо классификации) требует учёта тренда и масштаба, которые квантование может потерять.

6 Дополнительные исследования

6.1 Статистическая проверка различий

В этом подразделе результаты основных таблиц проверяются формальными тестами.

BTC: тривиальный baseline против обучаемых моделей

По таблицам 12 и 13 (разделы 5.6) для каждой пары (ticker, H) собираются $N = 8$ наблюдений RMSE. Применяется парный знаково-ранговый тест Уилкоксона (Wilcoxon signed-rank) для гипотезы «модель равна Naïve» в двустороннем варианте. Положительная разность означает, что модель *хуже* Naïve по RMSE.

Таблица 16 – Wilcoxon signed-rank test: каждая обучаемая модель против **Naive** на 8 конфигурациях BTC+ETH $\times \{H=5, 10, 15, 24\}$.

модель	медиана Δ RMSE	W	p-value	побед / поражений vs Naive
DLinear	+4,34	0	0,008	0/8
NHITS	+15,54	0	0,008	0/8
PatchTST	+3,69	0	0,008	0/8
TiDE	+3,85	0	0,008	0/8
TimesNet	+4,36	0	0,008	0/8

Содержательный вывод: **Naive** оказывается лучшей моделью на всех 8 проверенных конфигурациях для каждой из пяти обучаемых архитектур, p-value 0,008 — минимально достижимое значение при $N = 8$. Тест Фридмана для всех 6 моделей даёт $\chi^2 = 24,86$, $p = 0,0001$, отклоняя гипотезу об их эквивалентности; средний ранг **Naive** равен 1,00, у ближайшего конкурента TiDE — 3,00.

Это формально подтверждает основной тезис BTC-части работы: на криптовалютных рядах в проверенных настройках обучаемые модели не дают улучшения над тривиальным baseline по RMSE.

UCR: семь методов на 8 датасетах

По таблицам 6 и 8 собираются $N = 8$ наблюдений ассурасу на уникальных датасетах (Coffee, GunPoint, Trace, TwoLeadECG, ECG5000, FordA, Wafer, ElectricDevices) для 7 методов: ROCKET, MiniRocket, BOSS, SAX+Embed+Transformer, SAX-BoW+LogReg, Numeric PatchTST, Majority. Тест Фридмана даёт

$$\chi^2 = 42,00, \quad p = 1,84 \cdot 10^{-7}, \quad k = 7, \quad N = 8.$$

Гипотеза об эквивалентности всех семи методов отвергается с большим запасом. Средние ранги (1 = лучший на датасете):

ROCKET 1,69; MiniRocket 1,75; BOSS 2,56;
SAX+Embed 5,00; SAX-BoW 5,00; PatchTST 5,13; Majority 6,88.

Критическая разность Nemenyi при $\alpha = 0,05$, $k = 7$, $N = 8$:

$$CD = q_{0,05} \sqrt{\frac{k(k+1)}{6N}} = 2,949 \cdot \sqrt{\frac{56}{48}} \approx 3,19.$$

Различия признаются значимыми при превышении CD. Имеем три устойчивых вывода:

- три метода «верхней группы» (ROCKET, MiniRocket, BOSS) образуют один кластер: попарные разности рангов $\leq 0,87$, что меньше CD — между ними различия по этим $N = 8$ наблюдениям статистически не подтверждаются;

- каждый из трёх методов «верхней группы» значимо лучше любого из «нижней группы» (SAX+Embed, SAX-BoW, PatchTST, Majority): минимальная разность рангов $5,00 - 2,56 = 2,44$ для пары BOSS vs SAX+Embed — ниже CD; для пар ROCKET/MiniRocket vs SAX+/SAX-/PatchTST разности $\geq 3,25$ — значимо;
- Majority значимо хуже всех остальных шести методов.

Парные тесты Уилкоксона уточняют: MiniRocket vs SAX+Embed/SAX-BoW/PatchTST — $p = 0,004$, win/lose 8/0. MiniRocket vs BOSS — $p = 0,078$ (на грани), win/lose 5/1.

Содержательное резюме. На выбранной подвыборке UCR из 8 датасетов:

- три метода — ROCKET, MiniRocket, BOSS — статистически неотличимы друг от друга и значимо опережают остальные;
- BOSS — лучший среди именно *символьных* методов, но в верхней группе уступает в среднем random-convolutional методам ROCKET и MiniRocket;
- все три метода «верхней группы» значимо опережают как обучаемый Transformer на сыром численном сигнале (PatchTST), так и упрощённые символьные baseline (SAX+Embed, SAX-BoW);
- для более жёсткого упорядочивания внутри верхней группы нужно расширение выборки до полного UCR-128.

6.2 Вычислительная сложность

Один из практических аргументов в пользу методов «верхней группы» (BOSS, ROCKET, MiniRocket) против глубоких ансамблей — их вычислительная сложность.

Теоретические оценки.

- BOSS Ensemble: $O(n_{\text{train}} \cdot L \cdot |\mathcal{B}|)$ на обучении, где $|\mathcal{B}|$ — число базовых классификаторов с разными (w, ℓ, K) ; на инференсе $O(n_{\text{test}} \cdot n_{\text{train}})$ из-за 1-NN.
- ROCKET: $O(n_{\text{train}} \cdot L \cdot K_{\text{rocket}})$ на трансформации ($K_{\text{rocket}} \sim 10^4$ случайных ядер), затем линейная регрессия на матрице $n_{\text{train}} \times 2K_{\text{rocket}}$.
- MiniRocket: тот же порядок, но фиксированный детерминированный набор $K = 84$ ядер длины 9 и только один признак (ppv) на ядро. На практике в десятки раз быстрее ROCKET.
- PatchTST: $O(E \cdot n_{\text{train}} \cdot L^2 \cdot d_{\text{model}})$ при квадратичной сложности механизма внимания, где E — число эпох.
- InceptionTime: то же $O(E \cdot n_{\text{train}} \cdot L \cdot d^2)$ для одной сети, умножается на размер ансамбля (5 моделей).

Эмпирические измерения. Реальное время train+inference на каждом из 8 UCR-датасетов измерено в собственных прогонах (CPU; для PatchTST — GPU Tesla T4). Сводная таблица:

Таблица 17 – Эмпирическое время полного цикла обучение+инференс, секунды (CPU для BOSS/ROCKET/MiniRocket; GPU T4 для PatchTST). Длина ряда L и размер выборки указаны для контекста.

датасет	n_{train} / L	BOSS	ROCKET	MiniRocket
Coffee	28/286	38,2	7,5	14,6
GunPoint	50/150	11,1	3,1	0,3
Trace	100/275	39,4	5,7	0,5
TwoLeadECG	23/82	9,0	9,7	0,6
ECG5000	500/140	311,4	70,3	3,9
FordA	3601/500	34 020,2	256,3	17,0
Wafer	1000/152	784,8	110,5	6,0
ElectricDevices	8926/96	6 458,6	273,2	100,7
Сумма по 8	—	41 672,7	736,4	143,5
Сумма (часы)	—	$\approx 11,6$ ч	≈ 12 мин	$\approx 2,4$ мин

Содержательно:

- BOSS на крупных датасетах с длинными рядами катастрофически медленный. На FordA ($n_{\text{train}} = 3601$, $L = 500$) полный цикл занял **9 ч 22 мин** — ансамбль из десятков 1-NN классификаторов с разными (w, ℓ, K) на длинных рядах требует кубического времени относительно L для построения SFA-словарей и квадратичного для 1-NN-инференса.
- ROCKET в среднем в **57 раз быстрее** BOSS суммарно (736 с против 41 673 с).
- MiniRocket в **290 раз быстрее** BOSS и в 5 раз быстрее ROCKET.
- Опубликованные результаты InceptionTime требуют GPU и обучения ансамбля из 5 сетей — по порядку величины это часы на датасет (точные собственные измерения для InceptionTime в данной работе не проводились).

Вывод. Если цель — максимальная точность при минимальных вычислительных ресурсах, то на классификации временных рядов из выбранного диапазона предпочтителен **MiniRocket**: при сопоставимом или лучшем качестве он на два-три порядка быстрее BOSS и не требует GPU. BOSS остаётся ценен как лучший среди именно *символьных* методов и интерпретируемая альтернатива (через словарь SFA-слов), но не как метод выбора по соотношению «качество/время».

6.3 Краткий обзор WEASEL и HIVE-COTE

Помимо BOSS, в литературе по классификации временных рядов известны более сложные словарные и ансамблевые методы. Краткое описание двух наиболее значимых.

WEASEL. Метод Word ExtrAction for time SEries cLassification (WEASEL) [32] — развитие BOSS, в котором (i) используется не один, а несколько размеров скользящего окна одновременно; (ii) вместо 1-NN-классификатора используется логистическая регрессия с L1-регуляризацией; (iii) применяется χ^2 -отбор информативных слов. По опубликованным результатам, WEASEL обходит BOSS на части датасетов архива UCR при сопоставимой вычислительной сложности.

HIVE-COTE 2.0. Hierarchical Vote Collective of Transformation-based Ensembles (HIVE-COTE) и его актуальная версия HIVE-COTE 2.0 [33] — метаансамбль, объединяющий несколько разнородных классификаторов: словарные (BOSS/WEASEL), интервальные (TSF/RISE), shape-based (Shapelet Transform) и ROCKETS. Голосование производится взвешенно по предсказанной вероятности класса. На усреднённых результатах архива UCR-128 HIVE-COTE 2.0 является одним из формальных лидеров рейтинга [45, 34].

ROCKET. Метод RandOm Convolutional KErnel Transform [37] — быстрый недеревный метод, использующий случайные одномерные свёртки. Алгоритм генерирует $\sim 10^4$ случайных свёрточных ядер с произвольными длинами, dilation, padding, после чего для каждого входного ряда вычисляются два признака на ядро: *max* активации и *ppv* (proportion of positive values). Полученные $\sim 2 \cdot 10^4$ признаков подаются на линейный классификатор (Ridge или Logistic Regression). Несмотря на свою простоту, ROCKETS достигает state-of-the-art точности на архиве UCR при существенно меньших вычислительных затратах, чем HIVE-COTE и InceptionTime.

MiniRocket. Метод MiniRocket [38] — упрощение ROCKETS, в котором свёрточные ядра ограничены фиксированным набором (84 ядра длины 9 с дискретными dilation), а вычисляется только *ppv*-признак. Это делает MiniRocket почти полностью детерминированным и в десятки раз быстрее ROCKETS при сопоставимом качестве. На усреднённых результатах архива UCR MiniRocket конкурентоспособен с HIVE-COTE 2.0 и опережает InceptionTime по соотношению «качество/время».

В настоящей работе WEASEL и HIVE-COTE 2.0 в качестве собственных моделей не запускались (они упомянуты только для библиографической полноты как естественные следующие методы для расширения сравнения). **ROCKET** и **MiniRocket**, наоборот, были прогнаны как дополнительные сравнительные методы на тех же 8 датасетах UCR; результаты приведены в таблицах раздела 5 (6, 8, 10) и в анализе раздела 6.1.

Дополнительные методы feature-based классификации. Помимо словарных и random-convolutional методов, в литературе развивается feature-based направление: catch22 [39] извлекает 22 канонических временных признака, отобранных из библиотеки hctsa [40], и подаёт их в стандартный классификатор. Этот подход не использует ни нейросетей, ни символизации, и служит ещё одной альтернативной формой представления, не охваченной в основной части настоящей работы.

6.4 Foundation-модели для временных рядов как направление развития

Параллельно с глубокими специализированными архитектурами (PatchTST, TimesNet, TiDE) активно развивается направление *foundation-моделей для временных рядов*. Это большие предобученные модели, способные делать прогноз на новых временных рядах в режиме **zero-shot** без дообучения. Идейно они являются временным аналогом языковых моделей.

Chronos. Семейство моделей Chronos [24] от Amazon строится на архитектуре T5: ряд токенизируется через равномерное квантование значений, после чего модель обучается решать задачу авторегрессионного language modeling на смеси публичных и синтетических временных рядов. Версия Chronos-Bolt (использованная в наших экспериментах в режиме zero-shot) специально оптимизирована для скорости инференса. На дневной частоте BTC именно Chronos-Bolt-Small оказался наилучшей моделью по среднему RMSE (см. раздел 5.6).

TimesFM. Google-овский TimesFM [26] — decoder-only Transformer, обученный на корпусе из около 100 млрд точек реальных временных рядов. Архитектурно близок к языковым GPT-моделям, но работает с прямыми численными значениями без токенизации. Доступен для свободного использования.

Moirai. Salesforce-овский Moirai [27] — foundation-модель с явной поддержкой произвольной длины предсказания и гетерогенной частоты данных. Использует «masked time-series modeling» в духе BERT.

Lag-Llama. Lag-Llama [28] — декодер-only архитектура, основанная на лагированных признаках, обученная на крупном корпусе временных рядов в стиле next-token-prediction.

Связь с настоящей работой. В нашем расширенном бенчмарке на BTC (раздел 5.6) Chronos-Bolt-Small в режиме zero-shot оказывается одной из сильнейших моделей и на дневной частоте показывает лучший средний RMSE среди всех проверенных. Это эмпирически указывает направление, в котором имеет смысл двигаться дальше: вместо

обучения более сложных специализированных моделей на узком домене может быть продуктивно использовать большую предобученную модель в режиме zero-shot.

В разрезе основной темы работы — сравнения численного и символьного представлений — foundation-модели интересны тем, что они *внутри* себя выполняют именно символизацию входа: Chronos квантует ряд в дискретный токен из словаря размера ~ 4096 . Это можно рассматривать как ещё один аргумент в пользу того, что дискретное представление полезно даже на тех доменах, где явное применение классических символьных методов (BOSS) не работает — но реализуется оно не на уровне предобработки, а внутри обученной модели, поверх большого корпуса вспомогательных данных.

7 Ограничения работы

Чтобы выводы работы воспринимались корректно, необходимо явно перечислить ограничения проведённых экспериментов.

Объём UCR-выборки. Использованы 8 уникальных датасетов из 128, входящих в архив UCR. Они выбраны как стандартные тестовые наборы для словарных символьных методов (BOSS/WEASEL) и охватывают разные размеры обучающих выборок и длины рядов, но не являются репрезентативной случайной выборкой архива. Для строгого упорядочивания методов внутри «верхней группы» (BOSS, ROCKET, MiniRocket) необходимо расширение до полного UCR-128.

Финансовый домен ограничен криптовалютами. Эксперименты проведены на BTC/USDT и (в multi-step бенчмарке) на ETH/USDT. Криптовалютные ряды имеют специфические свойства (постоянная торговля 24/7, отсутствие официального закрытия, сильное влияние спекулятивного интереса), отличающие их от акций, валютных пар и товарных рынков. Перенос отрицательного результата на BTC на финансовые рынки в целом не проводится; для таких выводов требовался бы аналогичный эксперимент на акциях или валютных парах.

Гиперпараметры нейросетевых моделей. Гиперпараметры PatchTST, других моделей NeuralForecast и численного Transformer не оптимизировались исчерпывающе. Использовались разумные значения по умолчанию, увеличенные до 50 эпох и подключённый CosineAnnealing-планировщик в эксперименте по аугментации. Полный grid search или Bayesian-оптимизация могут улучшить эти модели; границы такого улучшения в работе не оценивались.

Сравнение с InceptionTime — ориентировочное. Значения ассигасу для InceptionTime взяты из публикации [34] и сайта [35], собственные прогоны InceptionTime в данной работе не проводились. Сравнение с собственными результатами BOSS, ROCKET

и MiniRocket, выполненными в едином протоколе **aeon**, носит ориентировочный характер и не может рассматриваться как полноценный контролируемый эксперимент.

Backtest упрощён. Контрольный backtest direction classification на BTC использует фиксированную комиссию 5 bps и проскальзывание 1 bp; не учитываются эффект ликвидности, рыночное воздействие, частичное исполнение, изменения комиссий по тарифным уровням, использование лимитных ордеров вместо taker. Даже с этими упрощениями активные стратегии оказываются убыточными — более реалистичная модель издержек только усилила бы этот вывод. Backtest следует рассматривать как качественную проверку «может ли направление с balanced accuracy 0,53–0,54 быть прибыльным», а не как оценку конкретной торговой стратегии.

Семейства MT5-данных — вспомогательный эксперимент. Внутренняя ablation numeric_mt5 / numeric_core / hybrid_mt5 / symbolic_* (раздел 5.6) проведена на отдельной выгрузке данных MetaTrader 5, а не на основном наборе из Binance Vision. Этот эксперимент использован как качественное подтверждение лидерства численных представлений на BTC, но не как часть основного бенчмарка с walk-forward валидацией. Соответствующие выводы маркированы в тексте как «вспомогательные».

Foundation-модели использованы только в zero-shot. Chronos Bolt Small применялся только в режиме zero-shot, без дообучения на BTC. Дообучение Chronos на финансовых данных (fine-tuning) могло бы улучшить результаты и не покрыто в данной работе.

Заключение

В работе систематически сравнены численные и символьные представления временных рядов в задачах прогнозирования и классификации на двух качественно различных доменах: финансовом ряде BTC/USDT и наборе из 8 уникальных датасетов архива UCR в 9 экспериментальных постановках (5 малых от 23 до 500 обучающих примеров и 4 крупных от 1000 до 8926; TwoLeadECG использован в двух режимах).

Основные результаты.

1. На малых датасетах UCR три метода ROCKET (1,3), MiniRocket (2,1) и BOSS (2,1) образуют верхнюю группу со статистически неотличимым качеством; BOSS — лучший среди именно символьных методов. Численный PatchTST оказывается худшим среди обучаемых на 4 из 5 наборов: 23–100 обучающих примеров недостаточно для обучения Transformer.
2. Аугментация PatchTST в пять раз (jittering, scaling, time warping) с расширением до 50 эпох и косинусным планировщиком сокращает разрыв с BOSS, но не закрывает

его. SAX+Embedding+Transformer без аугментации показывает в среднем то же качество, что аугментированный PatchTST: символизация на малой выборке играет роль, аналогичную data augmentation.

3. На крупных датасетах UCR (FordA, Wafer, ElectricDevices) MiniRocket занимает первое место со средним рангом 1,0, ROCKET — второе (2,0), BOSS — третье (3,3); все три метода «верхней группы» остаются выше PatchTST (4,3) и упрощённых символьных baseline. Это указывает на то, что random-convolutional методы лучше масштабируются по длине ряда, чем словарные, при сохранении общего вывода о полезности дискретного представления.
4. Контрольный прогон ROCKET и MiniRocket на тех же 8 датасетах показал, что в среднем по рангу random-convolutional методы (ROCKET: 1,69; MiniRocket: 1,75) превосходят BOSS (2,56); по тесту Фридмана $\chi^2 = 42,0$, $p = 1,8 \cdot 10^{-7}$. Все три метода «верхней группы» статистически неотличимы между собой и значительно опережают остальные. Собственные результаты BOSS, ROCKET и MiniRocket попадают в диапазон опубликованных значений InceptionTime; в частности, MiniRocket на ElectricDevices (0,753) даже превышает опубликованное значение InceptionTime (0,721).
5. По вычислительной стоимости MiniRocket на 2–3 порядка быстрее BOSS: суммарное время прогона на 8 датасетах — 143 с против 41 673 с (CPU). Это делает MiniRocket предпочтительным методом по соотношению «качество/время», тогда как BOSS остаётся ценен как лучший среди символьных и интерпретируемая альтернатива через явный словарь SFA-слов.
6. В шумовом тесте на ECG5000 BOSS лучше всего на чистых данных, но численный PatchTST с вниманием деградирует медленнее (−2,2% против −6,9% у BOSS при σ от 0 до 1,0). Гипотеза о шумоустойчивости символизации «самой по себе» не подтверждается; устойчивость определяется мощностью читателя.
7. На BTC ни одна модель не доминирует ни на одной частоте. На минутных данных лучшим оказывается PatchTST, на часовых — тривиальный Naive, на дневных — foundation-модель ChronosBoltSmall в режиме zero-shot. Во внутренней абляции 17 конфигураций `numeric_mt5` побеждает 9 раз, `numeric_core` — 5 раз, `symbolic_*` — ни разу как основное представление. Лучшие группы признаков — `candle_engineered` и `full_stack`.
8. Единственная конфигурация на BTC, в которой символьное представление в среднем по фолдам обходит и тривиальный baseline, и численный Transformer, — классификация направления на часовых данных при горизонте $h = 1$: `symbolic_no_volume` даёт balanced accuracy 0,538 против 0,500 у `majority_sign` и 0,524 у `numeric_relative`. Контрольный backtest показал, что соответствующий

статистический сигнал ($\text{hit rate} \approx 0,53\text{--}0,54$) не покрывает транзакционных издержек: даже при комиссии 5 bps активные стратегии (long/flat, long/short) на тесте теряют -55 и -76% соответственно, тогда как ZeroTrade-стратегия (cash, 0%) формально оказывается лучшей. Поэтому результат фиксируется как статистический, но не как практический.

Главный вывод. Полезность символьного представления временного ряда *существенно зависит от свойств домена*. На рядах с воспроизводимой локальной морфологией (UCR: ЭКГ, спектроскопия, жесты, показания датчиков) словарный символьный метод BOSS на выбранных наборах показывает значения ассигасы, сопоставимые с опубликованными результатами современных глубоких ансамблей, при существенно меньшей вычислительной стоимости. На финансовом ряде BTC, близком к процессу со случайным блужданием, в проведённых экспериментах на выбранных частотах и горизонтах не обнаружено статистически подтверждённого устойчивого преимущества обучаемых моделей — ни символьных, ни численных, включая foundation-модель Chronos Bolt Small и специализированные современные архитектуры прогнозирования временных рядов — над тривиальными baseline, за исключением отдельной конфигурации direction classification на 1h, $h = 1$. Это исключение относится только к статистической метрике balanced ассигасы и *не* подтверждается в экономическом backtest: с учётом транзакционных издержек активные стратегии оказываются убыточными, а ZeroTrade-стратегия (cash) формально остаётся лучшей.

Перспективы.

- Расширение UCR-сравнения до полного бенчмарка из 128 датасетов с целью количественно подтвердить наблюдение о доменной зависимости.
- Исследование совместного использования численного и символьного входов в одной модели (multi-stream Transformer): на BTC такая комбинация может оказаться полезной, поскольку численное представление BTC лучше численной модели, а символьное на $h = 1$, 1h формально обходит численную.
- Применение BOSS к финансовым данным после явной фильтрации режимов (low-volatility / high-volatility): вместо сравнения на всём ряде проверить, существуют ли подвыборки финансовых рядов, по поведению похожие на UCR.
- Расширение сравнения на BTC включением актуальных foundation-моделей (Chronos T5, TimesFM, Moirai) и точная количественная оценка их zero-shot преимущества.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Vaswani A., Shazeer N., Parmar N. et al. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 5998–6008. — URL: <https://arxiv.org/abs/1706.03762>.
- [2] Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems. — 2019. — Vol. 32. — P. 8024–8035.
- [3] Hyndman R. J., Athanasopoulos G. Forecasting: Principles and Practice. — 3rd ed. — Melbourne : OTexts, 2021. — 444 p. — URL: <https://otexts.com/fpp3/>.
- [4] Hyndman R. J., Koehler A. B. Another look at measures of forecast accuracy // International Journal of Forecasting. — 2006. — Vol. 22, № 4. — P. 679–688. — DOI: 10.1016/j.ijforecast.2006.03.001.
- [5] Diebold F. X., Mariano R. S. Comparing Predictive Accuracy // Journal of Business and Economic Statistics. — 1995. — Vol. 13, № 3. — P. 253–263.
- [6] Bergmeir C., Hyndman R. J., Koo B. A note on the validity of cross-validation for evaluating autoregressive time series prediction // Computational Statistics and Data Analysis. — 2018. — Vol. 120. — P. 70–83.
- [7] Makridakis S., Spiliotis E., Assimakopoulos V. The M4 Competition: 100,000 Time Series and 61 Forecasting Methods // International Journal of Forecasting. — 2020. — Vol. 36, № 1. — P. 54–74.
- [8] Fama E. F. Efficient Capital Markets: A Review of Theory and Empirical Work // The Journal of Finance. — 1970. — Vol. 25, № 2. — P. 383–417.
- [9] Samuelson P. A. Proof That Properly Anticipated Prices Fluctuate Randomly // Industrial Management Review. — 1965. — Vol. 6, № 2. — P. 41–49.
- [10] Brock W., Lakonishok J., LeBaron B. Simple Technical Trading Rules and the Stochastic Properties of Stock Returns // The Journal of Finance. — 1992. — Vol. 47, № 5. — P. 1731–1764.
- [11] Kristoufek L. BitCoin meets Google Trends and Wikipedia: Quantifying the relationship between phenomena of the Internet era // Scientific Reports. — 2013. — Vol. 3. — Article 3415. — DOI: 10.1038/srep03415.
- [12] Zhou H., Zhang S., Peng J. et al. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting // Proceedings of the AAAI Conference on Artificial Intelligence. — 2021. — Vol. 35, № 12. — P. 11106–11115. — arXiv:2012.07436.

- [13] Wu H., Xu J., Wang J., Long M. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting // Advances in Neural Information Processing Systems. — 2021. — Vol. 34. — P. 22419–22430. — arXiv:2106.13008.
- [14] Lim B., Arik S. O., Loeff N., Pfister T. Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting // International Journal of Forecasting. — 2021. — Vol. 37, № 4. — P. 1748–1764. — arXiv:1912.09363.
- [15] Nie Y., Nguyen N. H., Sinthong P., Kalagnanam J. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers // Proceedings of ICLR. — 2023. — arXiv:2211.14730.
- [16] Liu Y., Hu T., Zhang H. et al. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting // Proceedings of ICLR. — 2024. — arXiv:2310.06625.
- [17] Zeng A., Chen M., Zhang L., Xu Q. Are Transformers Effective for Time Series Forecasting? // Proceedings of the AAAI Conference on Artificial Intelligence. — 2023. — Vol. 37, № 9. — P. 11121–11128. — arXiv:2205.13504.
- [18] Challu C., Olivares K. G., Oreshkin B. N. et al. NHITS: Neural Hierarchical Interpolation for Time Series Forecasting // Proceedings of the AAAI Conference on Artificial Intelligence. — 2023. — Vol. 37, № 6. — P. 6989–6997. — arXiv:2201.12886.
- [19] Olivares K. G., Challu C., Marcjasz G. et al. Neural basis expansion analysis with exogenous variables (NBEATSx) // International Journal of Forecasting. — 2023. — Vol. 39, № 2. — P. 884–900.
- [20] Wu H., Hu T., Liu Y. et al. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis // Proceedings of ICLR. — 2023. — arXiv:2210.02186.
- [21] Das A., Kong W., Leach A. et al. Long-term Forecasting with TiDE: Time-series Dense Encoder. — arXiv:2304.08424. — 2023.
- [22] Olivares K. G., Challu C., Garza F. et al. NeuralForecast: User friendly state-of-the-art neural forecasting models. — PyCon Salt Lake City : Nixtla, 2022. — URL: <https://github.com/Nixtla/neuralforecast> (дата обращения: 26.04.2026).
- [23] Махова А. Г., Кумсков М. И., Арсланова А. Р. Исследование функций активации и гиперпараметров в архитектуре сетей долгой краткосрочной памяти для прогнозирования финансовых временных рядов. — М. : МГУ, 2024.
- [24] Ansari A. F., Stella L., Turkmen C. et al. Chronos: Learning the Language of Time Series. — arXiv:2403.07815. — 2024.
- [25] Chronos-Bolt model card [Электронный ресурс] : описание модели и release note. — Amazon Science, 2024. — URL: <https://huggingface.co/amazon/chronos-bolt-small> (дата обращения: 26.04.2026).

- [26] Das A., Kong W., Sen R., Zhou Y. A Decoder-Only Foundation Model for Time-Series Forecasting // Proceedings of the 41st International Conference on Machine Learning, PMLR. — 2024. — Vol. 235. — P. 10148–10167. — arXiv:2310.10688.
- [27] Woo G., Liu C., Kumar A., Xiong C., Savarese S., Sahoo D. Unified Training of Universal Time Series Forecasting Transformers (Moirai) // Proceedings of the 41st International Conference on Machine Learning, PMLR. — 2024. — arXiv:2402.02592.
- [28] Rasul K., Ashok A., Williams A. R. et al. Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting. — arXiv:2310.08278. — 2023.
- [29] Lin J., Keogh E., Lonardi S., Chiu B. A Symbolic Representation of Time Series, with Implications for Streaming Algorithms // Proceedings of the 8th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery. — New York : ACM, 2003. — P. 2–11.
- [30] Schäfer P., Höggqvist M. SFA: A Symbolic Fourier Approximation and Index for Similarity Search in High Dimensional Datasets // Proceedings of the 15th International Conference on Extending Database Technology (EDBT). — 2012. — P. 516–527.
- [31] Schäfer P. The BOSS is concerned with time series classification in the presence of noise // Data Mining and Knowledge Discovery. — 2015. — Vol. 29, № 6. — P. 1505–1530.
- [32] Schäfer P., Leser U. Fast and Accurate Time Series Classification with WEASEL // Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM). — 2017. — P. 637–646. — arXiv:1701.07681.
- [33] Middlehurst M., Large J., Flynn M., Lines J., Bagnall A. HIVE-COTE 2.0: a New Meta Ensemble for Time Series Classification // Machine Learning. — 2021. — Vol. 110, № 11. — P. 3211–3243. — arXiv:2104.07551.
- [34] Ismail Fawaz H., Lucas B., Forestier G. et al. InceptionTime: Finding AlexNet for Time Series Classification // Data Mining and Knowledge Discovery. — 2020. — Vol. 34, № 6. — P. 1936–1962. — arXiv:1909.04939.
- [35] Time Series Classification Website [Электронный ресурс] : репозиторий результатов для бенчмарков UCR/UEA. — URL: <https://timeseriesclassification.com> (дата обращения: 26.04.2026).
- [36] Bagnall A., Lines J., Bostrom A., Large J., Keogh E. The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances // Data Mining and Knowledge Discovery. — 2017. — Vol. 31, № 3. — P. 606–660. — DOI: 10.1007/s10618-016-0483-9.

- [37] Dempster A., Petitjean F., Webb G. I. ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels // Data Mining and Knowledge Discovery. — 2020. — Vol. 34, № 5. — P. 1454–1495. — DOI: 10.1007/s10618-020-00701-z.
- [38] Dempster A., Schmidt D. F., Webb G. I. MiniRocket: A Very Fast (Almost) Deterministic Transform for Time Series Classification // Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. — 2021. — P. 248–257. — DOI: 10.1145/3447548.3467231.
- [39] Lubba C. H., Sethi S. S., Knaute P., Schultz S. R., Fulcher B. D., Jones N. S. catch22: CAnonical Time-series CHaracteristics // Data Mining and Knowledge Discovery. — 2019. — Vol. 33, № 6. — P. 1821–1852.
- [40] Fulcher B. D., Jones N. S. Highly comparative feature-based time-series classification // IEEE Transactions on Knowledge and Data Engineering. — 2014. — Vol. 26, № 12. — P. 3026–3037.
- [41] Um T. T., Pfister F. M. J., Pichler D. et al. Data Augmentation of Wearable Sensor Data for Parkinson’s Disease Monitoring using Convolutional Neural Networks // Proceedings of the 19th ACM International Conference on Multimodal Interaction (ICMI). — 2017. — P. 216–220. — arXiv:1706.00527.
- [42] Middlehurst M., Ismail-Fawaz A., Guillaume A. et al. aeon: a Python Toolkit for Learning from Time Series // Journal of Machine Learning Research. — 2024. — Vol. 25. — P. 1–10.
- [43] Tavenard R., Faouzi J., Vandewiele G. et al. tslearn, A Machine Learning Toolkit for Time Series Data // Journal of Machine Learning Research. — 2020. — Vol. 21. — P. 1–6.
- [44] Faouzi J., Janati H. pyts: A Python Package for Time Series Classification // Journal of Machine Learning Research. — 2020. — Vol. 21. — P. 1–6.
- [45] Dau H. A., Bagnall A., Kamgar K. et al. The UCR Time Series Archive // IEEE/CAA Journal of Automatica Sinica. — 2019. — Vol. 6, № 6. — P. 1293–1305. — URL: https://www.cs.ucr.edu/~eamonn/time_series_data_2018/ (дата обращения: 26.04.2026).
- [46] Binance Public Data [Электронный ресурс] : исторические рыночные данные spot и futures. — URL: <https://data.binance.vision> (дата обращения: 26.04.2026).
- [47] yfinance [Электронный ресурс] : Python package for Yahoo Finance data. — URL: <https://pypi.org/project/yfinance/> (дата обращения: 26.04.2026).
- [48] TA-Lib: Technical Analysis Library, Python wrapper [Электронный ресурс]. — URL: <https://ta-lib.github.io/ta-lib-python/> (дата обращения: 26.04.2026).

А Репликационный пакет

В таблице 18 приведено соответствие между результатами курсовой работы и приложенными ноутбуками / файлами с числовыми данными. Все ноутбуки, исходные данные и CSV/JSON-файлы с числовыми результатами размещены в открытом репозитории: <https://github.com/qqsta37/symbolic-vs-numeric-timeseries>.

Таблица 18 – Соответствие таблиц и рисунков курсовой ноутбукам и файлам результатов. Номера таблиц/рисунков подставляются автоматически.

таблица рисунок	/ ноутбук / источник	файл с числами
Табл. 1: BTC — данные и протокол		описание в разделе 2.2
Табл. 2: — гиперпараметры Transformer		описание в разделе 3.2
Табл. 3: BTC регрессия (подтверждающий)	btc_transformer_ confirmatory_study.ipynb	res_final/ btc_transformer_ confirmatory_runs/ regression_summary.csv
Табл. 4: BTC направление (подтверждающий)	btc_transformer_ confirmatory_study.ipynb	direction_summary.csv, paper_ready_table.csv
Табл. 5: — гипотеза — эксперимент — результат		сводка по всем разделам
Табл. 6 и рис. 1: UCR малые	symbolic_vs_numeric_ coursework.ipynb, ucr_rocket_ minirocket.ipynb	cell outputs в ipynb; ucr_rocket_results.csv
Табл. 7 и рис. 2: PatchTST с аугментацией	fair_comparison.ipynb	cell outputs в ipynb
Табл. 8 и рис. 3: UCR крупные	fair_comparison.ipynb, ucr_rocket_only_large.ipynb	cell outputs в ipynb; ucr_rocket_results.csv
Табл. 9 и рис. 4: шумовой sweep	symbolic_vs_numeric_ coursework.ipynb	cell outputs в ipynb
Табл. 10: сравнение InceptionTime	публикация [34], [35]; собственные с прогоны	ucr_rocket_results.csv
Табл. 11: BTC SOTA регрессия	сводка из отчёта — Отчет_по_тестированию_BTC.pdf; исходные прогоны: btc_onestep_ neuralforecast.ipynb, btc_walkforward_sota.ipynb	
Табл. 12 и 13: BTC и ETH multi-step	btc_eth_multistep_ horizons.ipynb	bench_multistep_results.json, .csv
Табл. 14: BTC	btc transformer	direction_summary.csv

В Используемые версии библиотек

Для воспроизведения результатов работы существенны версии используемых программных библиотек. Эксперименты на BTC и ETH запускались на Kaggle и Colab с Python 3.11/3.12. Обучение нейросетевых моделей (PatchTST, TimesNet, NHITS, NBEATSx, DLinear, NLinear, TFT, TiDE, iTransformer) выполнено через библиотеку `NeuralForecast`; foundation-модель Chronos Bolt Small подгружалась в режиме zero-shot. Классификация на UCR (BOSS, ROCKET, MiniRocket) выполнена через библиотеку `aeon`; SAX — через `tslearn`, SAX-BoW — через `pyts`. Полный список ключевых версий приведён в таблице 19.

Таблица 19 – Версии ключевых библиотек на момент проведения экспериментов.

библиотека	версия
Python	3.11 / 3.12 (Kaggle/Colab)
PyTorch	2.10 (Kaggle T4 / P100); 2.x (Colab)
pytorch-lightning	2.x
neuralforecast	1.7.5
chronos-forecasting	актуальная на 04.2026
aeon	актуальная на 04.2026
tslearn	0.6.x
pyts	0.13.x
scikit-learn	1.3–1.5
numpy	1.26 / 2.0
pandas	2.0–2.3
scipy	1.11–1.13
matplotlib	3.x

С Список сокращений

BTC, ETH

биткойн, эфир (тикеры криптовалют)

UCR

UCR Time Series Archive

OHLCV

Open, High, Low, Close, Volume (свечной формат биржевых данных)

SAX

Symbolic Aggregate approXimation

SFA

Symbolic Fourier Approximation

BOSS

Bag-of-SFA-Symbols

LLM

Large Language Model

LoRA	Low-Rank Adaptation (метод параметр-эффективного дообучения)
PAA	Piecewise Aggregate Approximation
1-NN	one-nearest-neighbour classifier
TF-IDF	Term Frequency–Inverse Document Frequency
MAE, RMSE, MAPE, sMAPE, MASE	стандартные метрики ошибки прогноза
CD	Critical Difference (Nemenyi post-hoc test)
EMH	Efficient Market Hypothesis
ADF	Augmented Dickey–Fuller test
bp, bps	базисный пункт (1 bp = 0,01%), базисные пункты
DD	drawdown (просадка)